UNIVERSITÀ DEGLI STUDI DI PADOVA
CORSO DI LAUREA IN INFORMATICA (L-31)

Corso di Ingegneria del Software
Anno Accademico 2025/2026

Specifica Tecnica

Gruppo: NightPRO

Sistema: SmartOrder

swe.nightpro@gmail.com

Data: 2026-04-08
Versione: 0.6

Tabella delle Versioni

Versione	Data	Autore/i	Descrizione delle Modifiche	Verificatore
0.6	2026-04-08	Davide Biasuzzi	Aggiornamento dello stato di implementazione dei requisiti e allineamento alla rimozione di un requisito	Leonardo Bilato
0.5	2026-04-04	Francesco Zanella	Raffinamento dell'architettura (Backend e Frontend) per ottimizzare flussi asincroni e separazione dei servizi; approfondimento analitico della sezione Database. Aggiornato tabella requisiti	Leonardo Bilato
0.4	2026-04-02	G. Ponso, L. Bilato	Stesura sezione Architettura di dettaglio	M. M. Romascu
0.3	2026-03-31	D. Biasuzzi, F. Zanella, S. Perozzo	Riorganizzazione documento, inserimento dei diagrammi PlantUML e stesura delle sezioni 4, 5, 6 e 7	M. M. Romascu
0.2	2026-03-26	Davide Biasuzzi	Inserimento paragrafo su esecuzione e su tecnologie utilizzate	Leonardo Bilato
0.1	2026-03-20	Davide Biasuzzi	Stesura base documento (Introduzione e Requisiti di Sistema)	Leonardo Bilato

Indice

Componenti del Gruppo 9

1 Introduzione 10
1.1 Scopo del documento 10
1.2 Scopo del prodotto 10
1.3 Glossario 10
1.4 Riferimenti 10
1.4.1 Riferimenti normativi 10
1.4.2 Riferimenti informativi 10

2 Ambiente minimo 11
2.1 Software minimo 11
2.2 Hardware minimo 11

3 Tecnologie utilizzate 12
3.1 Linguaggi di programmazione 12
3.1.1 TypeScript 12
3.1.2 Python 12
3.1.3 SQL 12
3.2 Framework e librerie 12
3.2.1 Next.js 12
3.2.2 React 13
3.2.3 Tailwind CSS 13
3.2.4 FastAPI 13
3.2.5 Pydantic 13
3.2.6 OpenAI SDK 13
3.3 Strumenti di persistenza 13
3.3.1 PostgreSQL 13
3.3.2 psycopg2 14
3.4 Intelligenza artificiale 14
3.4.1 GPT-4o 14
3.4.2 GPT-4o-mini 14
3.4.3 Whisper 14
3.5 Infrastruttura e DevOps 14
3.5.1 Docker 14
3.5.2 Node.js 14
3.5.3 Uvicorn 15
3.5.4 GitHub Actions 15
3.5.5 ESLint 15

4 Architettura di sistema 16
4.1 Obiettivi architetturali 16
4.2 Vista C4 - Livello 1 (System Context) 16
4.3 Vista C4 - Livello 2 (Container) 17
4.4 Stile architetturale adottato 17

5 Architettura di deployment 19
5.1 Scenario di esecuzione 19
5.2 Nodi e responsabilità di deployment 19

5.3 Canali di comunicazione . . . 20
5.4 Aspetti operativi e CI/CD . . . 21

6 Design pattern **22**
6.1 Dependency Injection . . . 22
6.1.1 Descrizione del pattern . . . 22
6.1.2 Motivazioni dell'utilizzo del pattern . . . 22
6.1.3 Utilizzo del pattern nel progetto . . . 22
6.2 Adapter (Ports and Adapters) . . . 22
6.2.1 Descrizione del pattern . . . 22
6.2.2 Motivazioni dell'utilizzo del pattern . . . 22
6.2.3 Utilizzo del pattern nel progetto . . . 22
6.3 Repository Pattern . . . 23
6.3.1 Descrizione del pattern . . . 23
6.3.2 Motivazioni dell'utilizzo del pattern . . . 23
6.3.3 Utilizzo del pattern nel progetto . . . 23
6.4 Facade . . . 23
6.4.1 Descrizione del pattern . . . 23
6.4.2 Motivazioni dell'utilizzo del pattern . . . 23
6.4.3 Utilizzo del pattern nel progetto . . . 23
6.5 Observer (State Management Reattivo) . . . 23
6.5.1 Descrizione del pattern . . . 23
6.5.2 Motivazioni dell'utilizzo del pattern . . . 24
6.5.3 Utilizzo del pattern nel progetto . . . 24

7 Database **25**
7.1 Ruolo del database nel sistema . . . 25
7.2 Macro-aree informative . . . 25
7.3 Struttura delle principali tabelle . . . 26
7.3.0.1 Anagrafica Articoli (`anaart`) . . . 26
7.3.0.2 Sessioni e Chat (`chat_sessions`, `chat_messages`) . . . 26
7.3.0.3 Carrello (`cart_items`) . . . 26
7.3.0.4 Ordini Finalizzati (`orders`, `order_items`) . . . 26
7.3.0.5 Identità Temporanee e Persistenti (`app_users`, `anacli`, `asscli`) 27
7.4 Vincoli dati rilevanti . . . 27
7.5 Output verso ERP . . . 27

8 Architettura di dettaglio **28**
8.1 Backend - C4 Livello 4: Classi e interfacce . . . 28
8.1.1 Layer 1 - Controllers (Inbound Ports) . . . 28
8.1.1.1 AuthController . . . 28
8.1.1.2 ConversationController . . . 28
8.1.1.3 CartController . . . 29
8.1.1.4 OrderController . . . 29
8.1.1.5 TicketController . . . 30
8.1.1.6 FeedbackController . . . 30
8.1.1.7 ClientController . . . 31
8.1.1.8 HealthController . . . 31
8.1.1.9 ProductController . . . 31
8.1.1.10 RegisterController . . . 31
8.1.1.11 SSEController . . . 32

8.1.2 Layer 2 - Application Services (Use Cases) . . . 32
8.1.2.1 AuthService . . . 32
8.1.2.2 ConversationService . . . 32
8.1.2.3 CartService . . . 33
8.1.2.4 OrderService . . . 33
8.1.2.5 VectorSearchService . . . 34
8.1.2.6 TicketService . . . 34
8.1.2.7 FeedbackService . . . 34
8.1.2.8 ClientService . . . 35
8.1.2.9 ProductService . . . 35
8.1.2.10 SSEBroadcaster . . . 35
8.1.3 Layer 3 - Core Domain (Entities & Value Objects) . . . 35
8.1.3.1 Cliente . . . 35
8.1.3.2 Operatore . . . 36
8.1.3.3 Order . . . 36
8.1.3.4 Carrello . . . 37
8.1.3.5 RigaCarrello . . . 38
8.1.3.6 Session . . . 38
8.1.3.7 Ticket . . . 39
8.1.3.8 LogAI . . . 39
8.1.4 Layer 4 - Outbound Ports (Interfaces) . . . 40
8.1.4.1 IUserRepository . . . 40
8.1.4.2 ISessionManager . . . 41
8.1.4.3 IOrderRepository . . . 41
8.1.4.4 ITicketRepository . . . 41
8.1.4.5 ILogAIRepository . . . 42
8.1.4.6 IFileExporter . . . 42
8.1.4.7 IAIClient . . . 43
8.1.4.8 INotificationPort . . . 43
8.1.5 Layer 5 - Infrastructure Adapters . . . 44
8.1.5.1 PostgresAdapter . . . 44
8.1.5.2 OpenAIAdapter . . . 44
8.1.5.3 FileSystemAdapter . . . 44
8.1.5.4 WebSocketAdapter . . . 45
8.2 Frontend - C4 Livello 4 . . . 45
8.2.1 Layer 1 - Routing & Page Layer (Next.js App Router) . . . 45
8.2.1.1 RouteGuard . . . 45
8.2.1.2 AuthPages . . . 46
8.2.1.3 ClientePages . . . 46
8.2.1.4 BackofficePage . . . 47
8.2.1.5 SharedLayout . . . 48
8.2.2 Layer 2 - State Management Layer (React Context + Custom Hooks) . . . 48
8.2.2.1 AuthContext . . . 48
8.2.2.2 CartContext . . . 49
8.2.2.3 ChatContext . . . 49
8.2.2.4 NotificationContext . . . 50
8.2.2.5 CustomHooks . . . 50
8.2.2.6 RoleBasedRendering . . . 51
8.2.3 Layer 3 - Service Layer (API Client Modules REST/HTTPS + JWT) . . . 51
8.2.3.1 authService . . . 51

8.2.3.2 cartService . . . 52
8.2.3.3 productService . . . 52
8.2.3.4 orderService . . . 53
8.2.3.5 ticketService . . . 53
8.2.3.6 chatService . . . 53
8.2.3.7 feedbackService . . . 54
8.2.3.8 healthService . . . 54
8.2.3.9 sessionService . . . 54
8.2.4 Layer 4 - Real-time Layer (WebSocket / SSE Gateway) . . . 55
8.2.4.1 RTGateway . . . 55
8.2.4.2 ChatEventHandler . . . 55
8.2.4.3 TicketEventHandler . . . 56
8.2.4.4 OrderEventHandler . . . 56
8.2.4.5 AILogEventHandler . . . 57
8.2.5 Layer 5 - UI Component Library (Atomic Design) . . . 57
8.2.5.1 Atoms . . . 57
8.2.5.2 Molecules . . . 57
8.2.5.3 OrganismsCliente . . . 58
8.2.5.4 OrganismsOperatore . . . 58
8.2.5.5 SharedUIUtilities . . . 59

9 Stato dei requisiti funzionali **60**
9.1 Riepilogo stato requisiti prestazionali e di vincolo . . . 66

Elenco delle figure

1 Diagramma C4 L1 (PlantUML compilato): contesto del sistema SmartOrder. . . . 16
2 Diagramma C4 L2 (PlantUML compilato): vista container. . . . 17
3 Vista di deployment logico del sistema. . . . 19
4 UML della classe `AuthController`. . . . 28
5 UML della classe `ConversationController`. . . . 29
6 UML della classe `CartController`. . . . 29
7 UML della classe `OrderController`. . . . 29
8 UML della classe `TicketController`. . . . 30
9 UML della classe `FeedbackController`. . . . 30
10 UML della classe `ClientController`. . . . 31
11 UML della classe `HealthController`. . . . 31
12 UML della classe `ProductController`. . . . 31
13 UML della classe `RegisterController`. . . . 31
14 UML della classe `SSEController`. . . . 32
15 UML della classe `AuthService`. . . . 32
16 UML della classe `ConversationService`. . . . 33
17 UML della classe `CartService`. . . . 33
18 UML della classe `OrderService`. . . . 33
19 UML della classe `VectorSearchService`. . . . 34
20 UML della classe `TicketService`. . . . 34
21 UML della classe `FeedbackService`. . . . 35
22 UML della classe `ClientService`. . . . 35
23 UML della classe `ProductService`. . . . 35
24 UML della classe `SSEBroadcaster`. . . . 35

25 UML della classe `Cliente`. 36
26 UML della classe `Operatore`. 36
27 UML della classe `Order`. 37
28 UML della classe `Carrello`. 37
29 UML della classe `RigaCarrello`. 38
30 UML della classe `Session`. 39
31 UML della classe `Ticket`. 39
32 UML della classe `LogAI`. 40
33 UML dell'interfaccia `IUserRepository`. 40
34 UML dell'interfaccia `ISessionManager`. 41
35 UML dell'interfaccia `IOrderRepository`. 41
36 UML dell'interfaccia `ITicketRepository`. 42
37 UML dell'interfaccia `ILogAIRepository`. 42
38 UML dell'interfaccia `IFileExporter`. 43
39 UML dell'interfaccia `IAIClient`. 43
40 UML dell'interfaccia `INotificationPort`. 43
41 UML della classe `PostgresAdapter`. 44
42 UML della classe `OpenAIAdapter`. 44
43 UML della classe `FileSystemAdapter`. 45
44 UML della classe `WebSocketAdapter`. 45
45 UML del componente `RouteGuard`. 46
46 UML del componente `AuthPages`. 46
47 UML del componente `ClientePages`. 47
48 UML del componente `BackofficePage`. 47
49 UML del componente `SharedLayout`. 48
50 UML del componente `AuthContext`. 48
51 UML del componente `CartContext`. 49
52 UML del componente `ChatContext`. 50
53 UML del componente `NotificationContext`. 50
54 UML del componente `CustomHooks`. 51
55 UML del componente `RoleBasedRendering`. 51
56 UML del componente `authService`. 52
57 UML del componente `cartService`. 52
58 UML del componente `productService`. 52
59 UML del componente `orderService`. 53
60 UML del componente `ticketService`. 53
61 UML del componente `chatService`. 54
62 UML del componente `feedbackService`. 54
63 UML del componente `healthService`. 54
64 UML del componente `sessionService`. 55
65 UML del componente `RTGateway`. 55
66 UML del componente `ChatEventHandler`. 56
67 UML del componente `TicketEventHandler`. 56
68 UML del componente `OrderEventHandler`. 56
69 UML del componente `AILogEventHandler`. 57
70 UML del componente `Atoms`. 57
71 UML del componente `Molecules`. 58
72 UML del componente `OrganismsCliente`. 58
73 UML del componente `OrganismsOperatore`. 59
74 UML del componente `SharedUIUtilities`. 59

Elenco delle tabelle

1 Componenti del gruppo NightPRO. 9
2 Software minimo per il deployment. 11
3 Hardware minimo richiesto. 11
4 C4 L1 - Contesto del sistema SmartOrder. 16
5 C4 L2 - Container e responsabilita. 17
6 Nodi logici di deployment e relative responsabilità. 20
7 Aree logiche del modello dati relazionale. 25
8 Stato dei requisiti funzionali (fonte: Analisi dei Requisiti PB v2.2) 66
9 Riepilogo implementazione requisiti prestazionali e di vincolo 66

Componenti del Gruppo

Cognome	**Nome**	**Matricola**
Biasuzzi	Davide	2111000
Bilato	Leonardo	2071084
Zanella	Francesco	2116442
Romascu	Mihaela-Mariana	2079726
Ogniben	Michele	2042325
Perozzo	Samuele	2110989
Ponso	Giovanni	2000558

Tabella 1: Componenti del gruppo NightPRO.

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di descrivere l'architettura software del sistema *SmartOrder*, definendo le scelte tecnologiche, i pattern architetturali e progettuali adottati, e l'architettura di dettaglio di ciascun componente. La Specifica Tecnica costituisce il riferimento principale per la fase di codifica, garantendo coerenza tra progettazione e implementazione.

1.2 Scopo del prodotto

SmartOrder è una piattaforma B2B per l'automazione della gestione degli ordini commerciali. Il sistema consente ai clienti di formulare ordini in linguaggio naturale (testo, audio, immagini) e li converte automaticamente in ordini strutturati pronti per l'integrazione con sistemi ERP aziendali, riducendo errori e tempi di inserimento manuale.

1.3 Glossario

Per evitare ambiguità, i termini tecnici e di dominio utilizzati nel documento sono definiti nel *Glossario v2.0*. I termini presenti nel glossario sono contrassegnati con il simbolo $_{\text{G}}$ alla prima occorrenza in ogni sezione.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- Norme di Progetto v2.0 (Ultima modifica 2026-03-20)

1.4.2 Riferimenti informativi

- Glossario v2.0 (Ultima modifica 2026-03-20)
- Piano di Qualifica v1.0 (Ultima modifica 2026-03-20)
- Next.js Documentation (Ultima modifica 2026-03-20)
- FastAPI Documentation (Ultima modifica 2026-03-20)
- OpenAI API Documentation (Ultima modifica 2026-03-20)
- PostgreSQL Documentation (Ultima modifica 2026-03-20)
- Software Architecture Patterns, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Dependency Injection, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Design Pattern Creazionali, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Design Pattern Strutturali, SWE 2025-2026 (Ultima modifica 2026-03-26)

2 Ambiente minimo

2.1 Software minimo

Software	Descrizione	Versione minima
Node.js	Runtime JavaScript per l'esecuzione dell'applicazione	$\geq$ 20.0
npm	Gestore pacchetti Node.js	$\geq$ 10.0
Python	Interprete Python per il modulo AI	$\geq$ 3.11
pip	Gestore pacchetti Python	$\geq$ 23.0
PostgreSQL	Database relazionale	$\geq$ 16
Browser web	Chrome, Firefox, Safari, Edge	Ultime 2 versioni

Tabella 2: Software minimo per il deployment.

2.2 Hardware minimo

Componente	Requisito
CPU	Processore 64-bit, $\geq$ 2 core
RAM	$\geq$ 4 GB
Spazio disco	$\geq$ 5 GB
Connessione di rete	Richiesta per accesso alle API OpenAI

Tabella 3: Hardware minimo richiesto.

3 Tecnologie utilizzate

In questa sezione vengono elencate le tecnologie adottate per lo sviluppo del sistema *SmartOrder*. Per ciascuna tecnologia viene indicata la versione utilizzata, la motivazione della scelta e il contesto d'impiego nel progetto.

3.1 Linguaggi di programmazione

3.1.1 TypeScript

- **Versione:** 5
- **Documentazione ufficiale:** TypeScript Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Frontend
- **Contesto di utilizzo:** linguaggio principale per l'interfaccia utente dell'applicativo Next.js, sia per i componenti React che per la gestione dello stato lato client.
- **Vantaggi:** tipizzazione statica che riduce errori a runtime, ottimo supporto IDE, compatibilità nativa con React e Next.js.
- **Svantaggi:** overhead di compilazione rispetto a JavaScript puro.

3.1.2 Python

- **Versione:** 3.11
- **Documentazione ufficiale:** Python 3 Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** implementazione della pipeline di elaborazione degli ordini tramite intelligenza artificiale e gestione delle API REST tramite FastAPI.
- **Vantaggi:** ecosistema maturo in ambito NLP/ML, compatibilità nativa con le librerie OpenAI, sintassi concisa, supporto asincrono con Uvicorn/ASGI.
- **Svantaggi:** prestazioni inferiori a linguaggi compilati; mitigato dall'uso di Uvicorn (ASGI asincrono).

3.1.3 SQL

- **Documentazione ufficiale:** ISO/IEC 9075:2023 (SQL) (Ultima modifica 2026-03-26)
- **Ambito:** Database
- **Contesto di utilizzo:** interrogazioni sul catalogo prodotti, gestione ordini, sessioni e carrelli.

3.2 Framework e librerie

3.2.1 Next.js

- **Versione:** 15.5
- **Documentazione ufficiale:** Next.js Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Frontend full-stack
- **Contesto di utilizzo:** framework principale per il frontend. Gestisce il rendering dell'interfaccia utente (React) e la logica lato client.
- **Vantaggi:** App Router con Server Components, SSR e SSG, integrazione ottimale con React.
- **Svantaggi:** curva di apprendimento per le convenzioni App Router; vincolo a React come libreria UI.

3.2.2 React

- **Versione:** 19
- **Documentazione ufficiale:** React Documentation (Ultima modifica 2026-03-26)
- **Ambito:** UI
- **Contesto di utilizzo:** costruzione dell'interfaccia utente tramite componenti dichiarativi, gestione dello stato con Context API e Hooks.
- **Vantaggi:** ecosistema vastissimo, modello a componenti riutilizzabili, Virtual DOM per aggiornamenti efficienti.

3.2.3 Tailwind CSS

- **Versione:** 4.1
- **Documentazione ufficiale:** Tailwind CSS Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Styling
- **Contesto di utilizzo:** stilizzazione dei componenti React con approccio utility-first.
- **Vantaggi:** prototipazione rapida, design responsive$_G$ integrato, nessun CSS custom da mantenere.
- **Svantaggi:** classi verbose nel markup; mitigato dalla composizione in componenti React.

3.2.4 FastAPI

- **Versione:** 0.115
- **Documentazione ufficiale:** FastAPI Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** espone gli endpoint `/chat` e `/transcribe` per l'elaborazione AI, gestione degli ordini e delle sessioni. Funziona come backend principale dell'applicazione.
- **Vantaggi:** validazione automatica dei dati tramite Pydantic$_G$, documentazione OpenAPI auto-generata, supporto nativo async/await.
- **Svantaggi:** richiede un processo separato (Uvicorn) rispetto al runtime Node.js.

3.2.5 Pydantic

- **Versione:** 2.9
- **Documentazione ufficiale:** Pydantic Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** definizione dei modelli di request/response per le API FastAPI, con validazione automatica dei tipi.

3.2.6 OpenAI SDK

- **Versione:** 1.51
- **Documentazione ufficiale:** OpenAI API Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** SDK$_G$ ufficiale per l'integrazione con GPT-4o, GPT-4o-mini e Whisper.

3.3 Strumenti di persistenza

3.3.1 PostgreSQL

- **Versione:** 16
- **Documentazione ufficiale:** PostgreSQL 16 Documentation (Ultima modifica 2026-03-26)

- **Ambito:** Database
- **Contesto di utilizzo:** unica base dati del sistema, accessibile dal modulo AI tramite Python (`psycopg2`). Tutte le operazioni sul catalogo prodotti, ordini, sessioni e carrelli avvengono tramite FastAPI.
- **Vantaggi:** robustezza, supporto a query complesse e full-text search, estensibilità (pgvector per Vector Search$_G$).

3.3.2 psycopg2

- **Documentazione ufficiale:** psycopg2 Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Modulo AI / Backend
- **Contesto di utilizzo:** driver Python per PostgreSQL con connection pooling$_G$ (`SimpleConnectionPool`).

3.4 Intelligenza artificiale

3.4.1 GPT-4o

- **Versione:** 2024-08-06
- **Documentazione ufficiale:** OpenAI GPT-4o (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** modello linguistico principale per la comprensione degli ordini, la classificazione degli intenti e le decisioni di business. Utilizzato con temperature=0.1 per risposte deterministiche.

3.4.2 GPT-4o-mini

- **Documentazione ufficiale:** OpenAI GPT-4o-mini (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** modello leggero per l'estrazione rapida dei parametri di ricerca e la classificazione semantica degli intenti utente.

3.4.3 Whisper

- **Versione:** 1
- **Documentazione ufficiale:** OpenAI Speech-to-Text (Whisper) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** trascrizione di messaggi audio in testo, con prompt specifico per il dominio Ergon e lingua forzata a italiano.

3.5 Infrastruttura e DevOps

3.5.1 Docker

- **Versione:** $\geq$ 24.0
- **Documentazione ufficiale:** Docker Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** containerizzazione dell'intero sistema per garantire portabilità e riproducibilità degli ambienti. Docker Compose ($\geq$ 2.20) orchestra l'avvio dei container.

3.5.2 Node.js

- **Versione:** 20
- **Documentazione ufficiale:** Node.js 20 Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** runtime JavaScript per l'esecuzione di Next.js in sviluppo e produzione.

3.5.3 Uvicorn

- **Versione:** 0.32
- **Documentazione ufficiale:** Uvicorn Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** server ASGI$_G$ per l'esecuzione di FastAPI, con supporto a richieste asincrone.

3.5.4 GitHub Actions

- **Documentazione ufficiale:** GitHub Actions Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** pipeline di integrazione continua per quality checks (ESLint, build verification).

3.5.5 ESLint

- **Versione:** 9
- **Documentazione ufficiale:** ESLint Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** linter$_G$ statico per il codice TypeScript/JavaScript.

4 Architettura di sistema

Questa sezione illustra l'architettura ad alto livello del sistema SmartOrder. Vengono definiti gli obiettivi progettuali che hanno guidato le scelte tecnologiche e viene fornita una rappresentazione progressiva del sistema utilizzando il modello C4, partendo dal contesto generale fino ad arrivare ai container principali. Infine, viene motivato lo stile architetturale adottato.

4.1 Obiettivi architetturali

L'architettura del sistema *SmartOrder* e stata definita per soddisfare i seguenti obiettivi:

- separare in modo netto responsabilita di presentazione, logica applicativa e persistenza;
- mantenere il backend indipendente dalla tecnologia client;
- supportare il flusso *Human-in-the-Loop* per i casi a bassa affidabilita AI;
- garantire integrazione con ERP tramite output JSON su cartella condivisa;
- favorire estendibilita verso nuovi canali di input (es. immagini, canali esterni).

4.2 Vista C4 - Livello 1 (System Context)

Il diagramma C4 Livello 1 individua i confini del sistema e gli attori esterni principali.

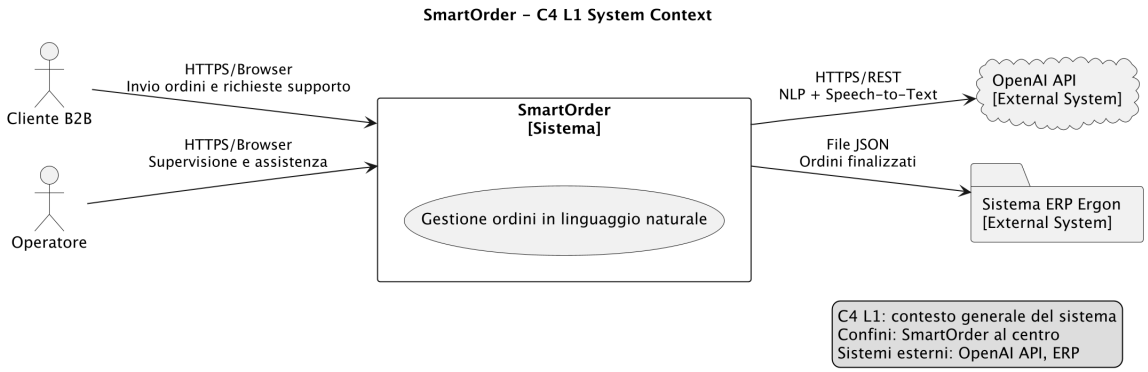

Figura 1: Diagramma C4 L1 (PlantUML compilato): contesto del sistema SmartOrder.

Elemento	Ruolo nel contesto
Cliente	Agente commerciale B2B che inserisce ordini in linguaggio naturale e richiede assistenza.
Operatore	Utente interno che supervisiona ticket, casi a bassa confidence e correzioni AI.
SmartOrder	Sistema oggetto della specifica; acquisisce input multimodale e genera ordini strutturati.
OpenAI API	Servizio esterno usato per comprensione linguaggio naturale e trascrizione audio.
Sistema ERP	Gestionale aziendale che riceve ordini finalizzati in formato JSON.

Tabella 4: C4 L1 - Contesto del sistema SmartOrder.

4.3 Vista C4 - Livello 2 (Container)

Il diagramma C4 Livello 2 (pagina `C4L2`) decompone SmartOrder nei container applicativi principali.

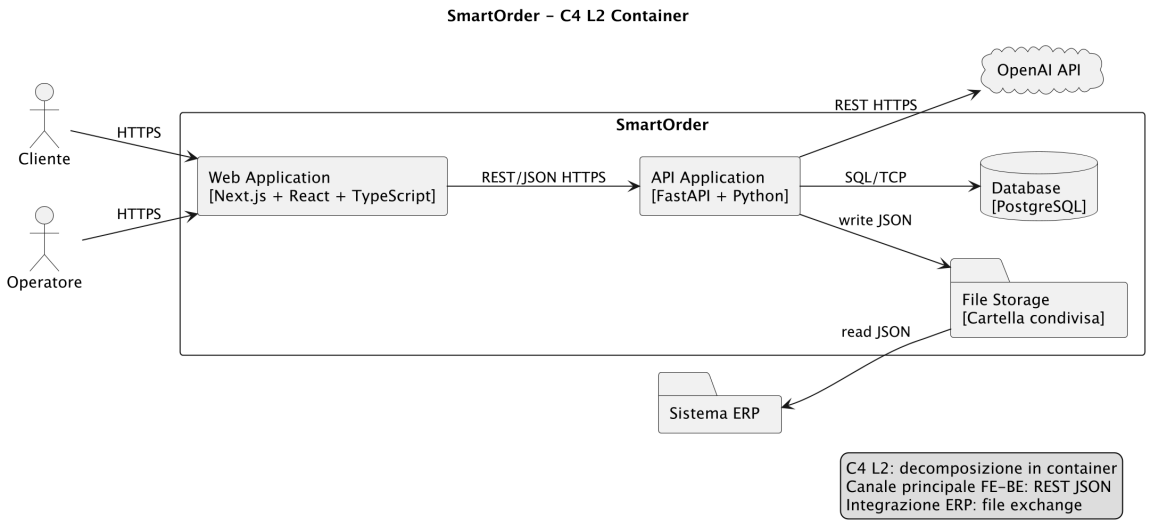

Figura 2: Diagramma C4 L2 (PlantUML compilato): vista container.

Container	Responsabilita
Web Application (Next.js + React + TypeScript)	Interfaccia cliente/operatore, gestione sessione utente, rendering chat, carrello, storico, dashboard operative.
API Application (FastAPI)	Business logic di dominio: autenticazione, orchestrazione AI, gestione carrello/ordini/ticket, persistenza e pubblicazione output.
Database (PostgreSQL)	Persistenza strutturata di utenti, sessioni, carrelli, ordini, ticket, log AI e catalogo.
File Storage (cartella condivisa)	Punto di consegna ordini finalizzati in JSON verso processo ERP esterno.

Tabella 5: C4 L2 - Container e responsabilita.

4.4 Stile architetturale adottato

La soluzione adotta uno stile **client-server a servizi**, con frontend e backend separati e API REST come contratto principale. All'interno del backend viene applicata una suddivisione per layer logici (Controller, Application Service, Core Domain, Outbound Ports) coerente con i principi di *Ports and Adapters*.

Le principali motivazioni della scelta sono:

- isolamento della logica di business da dipendenze infrastrutturali;
- testabilita delle regole applicative in assenza di interfaccia grafica;

- sostituibilita di adapter esterni (servizi AI, storage, connettori DB) con impatto ridotto;
- evoluzione indipendente dei rilasci frontend e backend.

5 Architettura di deployment

Questa sezione descrive la distribuzione fisica dei componenti del sistema *SmartOrder*, illustrando gli ambienti di esecuzione, i nodi infrastrutturali coinvolti e le strategie di Continuous Integration e Continuous Deployment (CI/CD) adottate.

5.1 Scenario di esecuzione

Il sistema prevede due configurazioni principali per l'esecuzione, pensate per separare l'ambiente di sviluppo da quello di produzione:

- **Ambiente di Sviluppo (Locale)**: i servizi di frontend e backend vengono eseguiti localmente sulla macchina di sviluppo, sfruttando la containerizzazione (es. Docker Compose) o task eseguiti in locale per avviare l'applicativo e un database PostgreSQL isolato. Questo garantisce un ambiente di test riproducibile.
- **Ambiente di Produzione (Cloud)**: l'intero applicativo è ospitato sulla piattaforma cloud **Railway**. Railway gestisce nativamente il provisioning delle risorse, ospitando in container isolati i servizi di frontend (Next.js), backend (FastAPI) e il database relazionale (PostgreSQL).

5.2 Nodi e responsabilità di deployment

La seguente vista descrive la mappatura tra i container software e le risorse cloud che li ospitano in ambiente di produzione.

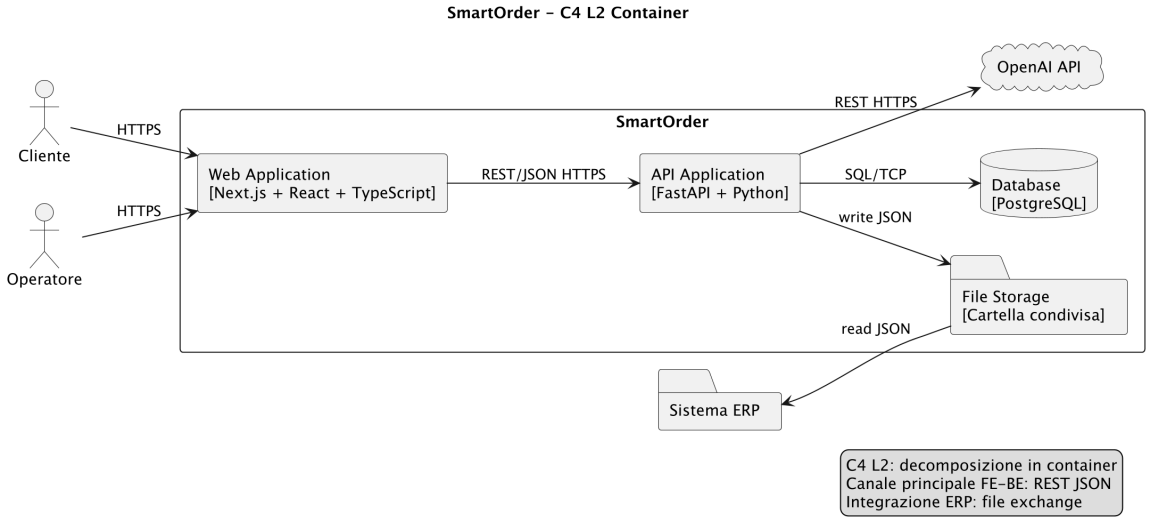

Figura 3: Vista di deployment logico del sistema.

Nodo Cloud	Responsabilità
Frontend Service(Railway)	Espone la web application in Next.js tramite protocollo HTTPS. Gestisce il rendering dell'interfaccia utente, l'idratazione lato client e inoltra le richieste API verso il backend.
Backend Service(Railway)	Esegue il server Uvicorn con l'applicazione FastAPI. Centralizza la logica di business, l'orchestrazione delle chiamate AI e la validazione transazionale degli ordini.
Database Service(Railway PostgreSQL)	Istanza cloud gestita di PostgreSQL. Conserva i dati transazionali (utenti, ordini, carrelli, ticket) garantendo integrità, persistenza e backup continui.
File Storage Condiviso(Object Storage S3)	Bucket cloud (es. Amazon S3 o compatibile) utilizzato come buffer di integrazione aziendale. Il backend vi deposita i file JSON degli ordini finalizzati, i quali vengono successivamente prelevati dal sistema ERP esterno.
OpenAI Cloud API(Servizio Esterno)	Endpoint gestiti direttamente da OpenAI, richiamati dal backend per i servizi di Natural Language Processing (GPT-4o) e di speech-to-text (Whisper).

Tabella 6: Nodi logici di deployment e relative responsabilità.

Nota architetturale sullo Storage Condiviso: Per l'integrazione con l'ERP aziendale si è scelto di adottare un **Object Storage**. Rispetto a una semplice cartella di rete, un bucket cloud offre standard di sicurezza enterprise, garantisce isolamento dei permessi e fornisce un facile accesso tramite API per il polling da parte di sistemi terzi.

5.3 Canali di comunicazione

Le comunicazioni tra i vari nodi del sistema avvengono esclusivamente tramite protocolli sicuri e standardizzati:

- **Frontend ↔ Backend**: le chiamate API avvengono su canale cifrato HTTPS, scambiando payload in formato JSON. L'autenticazione è garantita tramite token o cookie di sessione sicuri.
- **Backend ↔ PostgreSQL**: connessione su protocollo TCP proprietario del database, ottimizzata tramite un meccanismo di *connection pooling* (gestito tramite `psycopg2`) per sopportare picchi di traffico.
- **Backend ↔ OpenAI API**: invocazioni REST su rete pubblica cifrata HTTPS verso i server cloud di OpenAI.
- **Backend → File Storage**: scritture nel bucket cloud tramite chiamate API autenticate, necessarie a depositare l'ordine json per il sistema ERP.

5.4 Aspetti operativi e CI/CD

Per garantire rilasci continui, controllati e privi di disservizi, il progetto adotta pratiche automatizzate di *Continuous Integration* e *Continuous Deployment* (CI/CD):

- **Continuous Integration (CI)**: grazie a *GitHub Actions*, all'apertura di ogni Pull Request o push sui rami principali viene triggerato un workflow di controllo qualità (es. linter tramite ESLint) per prevenire il rilascio di codice formalmente scorretto.
- **Continuous Deployment (CD) automatizzato**: il deployment in produzione è delegato a **Railway**, il quale è interfacciato direttamente al repository GitHub. Ad ogni push sul branch principale, Railway rileva automaticamente i cambiamenti e lancia il processo di build e deploy. Questo avviene in modalità *zero-downtime*, aggiornando i container senza causare interruzioni agli utenti.
- **Gestione dei Segreti**: chiavi API, credenziali del database e secret di cifratura non vengono mai committati nel repository. Sono configurati come variabili d'ambiente (Environment Variables) criptate direttamente all'interno della dashboard del progetto su Railway.

6 Design pattern

Questa sezione descrive i principali pattern architetturali e progettuali (ispirati alle soluzioni della *Gang of Four* e ai pattern enterprise) adottati per risolvere problemi ricorrenti nel sistema *SmartOrder*, sia lato backend che lato frontend.

6.1 Dependency Injection

6.1.1 Descrizione del pattern

La Dependency Injection è un pattern strutturale che implementa l'Inversione del Controllo (IoC). Consiste nel fornire le dipendenze (oggetti o servizi) a una classe dall'esterno, anziché lasciare che sia la classe stessa a istanziarle internamente.

6.1.2 Motivazioni dell'utilizzo del pattern

L'obiettivo principale è il disaccoppiamento dei componenti. Iniettando le dipendenze si rispetta il principio di Inversione delle Dipendenze (la "D" dei principi SOLID), rendendo il codice estremamente più modulare. Questo risulta fondamentale per la testabilità del sistema: permette infatti di iniettare agilmente dei *mock* o *stub* al posto dei servizi reali durante l'esecuzione degli unit test.

6.1.3 Utilizzo del pattern nel progetto

Il pattern è applicato in modo sistematico nel backend FastAPI. I controller del Livello 1 (Inbound Ports), come il `ConversationController` o il `CartController`, non creano mai direttamente le istanze dei service. Al contrario, ricevono i servizi necessari (es. `ConversationService`, `CartService`) tramite il meccanismo nativo di iniezione delle dipendenze del framework.

6.2 Adapter (Ports and Adapters)

6.2.1 Descrizione del pattern

L'Adapter è un pattern strutturale che permette a due interfacce incompatibili di collaborare. Agisce come un "traduttore" o un "ponte", incapsulando la logica di un sistema esterno e offrendo un'interfaccia compatibile con il dominio interno dell'applicazione.

6.2.2 Motivazioni dell'utilizzo del pattern

In un'architettura di tipo *Ports and Adapters* (Architettura Esagonale), questo pattern è cruciale per isolare la logica di business (il "Core") dalle dipendenze tecnologiche esterne. Evita che i dettagli di librerie di terze parti o SDK "inquinino" il dominio applicativo, permettendo di sostituire un fornitore di servizi in futuro modificando solo l'Adapter, senza toccare le regole di business.

6.2.3 Utilizzo del pattern nel progetto

Nel backend, il pattern si concretizza nel Livello 5 (Infrastructure Adapters). Interfacce come `IAIClient` fungono da porte in uscita (Outbound Ports). La classe `OpenAIAdapter` implementa questa interfaccia e agisce come Adapter verso le API esterne, traducendo le entità del dominio di SmartOrder in formato JSON/REST specifico richiesto dai server di OpenAI, proteggendo il resto del sistema dai dettagli della libreria esterna.

6.3 Repository Pattern

6.3.1 Descrizione del pattern

Il Repository è un pattern architetturale (di livello enterprise) che media tra la logica di dominio e il layer di data mapping. Fornisce un'interfaccia che simula una collezione di oggetti in memoria, nascondendo le complessità legate all'estrazione e al salvataggio dei dati persistenti.

6.3.2 Motivazioni dell'utilizzo del pattern

L'utilizzo del Repository centralizza l'accesso al database, evitando la frammentazione delle query SQL all'interno della logica di business. Garantisce un singolo punto di verità per le operazioni CRUD, semplifica la manutenzione del codice legato alla persistenza e facilita eventuali migrazioni tecnologiche.

6.3.3 Utilizzo del pattern nel progetto

Nel backend, l'interazione con il database relazionale (PostgreSQL) è astratta attraverso le interfacce definite nel Livello 4 (Outbound Ports) come `IUserRepository`, `IOrderRepository` e `ITicketRepository`. L'implementazione pratica di queste interfacce è delegata all'infrastruttura sottostante (come `PostgresAdapter` o `ISessionManager`), in modo che i servizi applicativi lavorino unicamente con oggetti di dominio.

6.4 Facade

6.4.1 Descrizione del pattern

Il Facade è un pattern strutturale che fornisce un'interfaccia unificata, astratta e semplificata a un insieme di interfacce o moduli più complessi all'interno di un sottosistema, nascondendone i dettagli di basso livello.

6.4.2 Motivazioni dell'utilizzo del pattern

Il pattern riduce l'accoppiamento tra i client e il sottosistema complesso, offrendo un punto di accesso centralizzato e facile da usare. Questo migliora la leggibilità del codice, agevola lo sviluppo e riduce la curva di apprendimento per i componenti che devono interfacciarsi con logiche articolate.

6.4.3 Utilizzo del pattern nel progetto

Nel frontend React/Next.js, i moduli presenti nel Livello 3 (Service Layer API Client) fungono da Facade verso la rete: `authService`, `cartService`, `productService`, `orderService`, `ticketService`, `chatService` e `feedbackService`. Questi moduli nascondono ai componenti React la complessità delle comunicazioni HTTP/REST verso FastAPI (token JWT, serializzazione JSON, gestione errori), offrendo un'interfaccia uniforme e semplice da invocare.

6.5 Observer (State Management Reattivo)

6.5.1 Descrizione del pattern

L'Observer è un pattern comportamentale che definisce una dipendenza uno-a-molti tra oggetti. Quando l'oggetto osservato (Subject) cambia stato, tutti gli oggetti dipendenti (Observer) vengono notificati e aggiornati automaticamente.

6.5.2 Motivazioni dell'utilizzo del pattern

Nel contesto dello sviluppo di interfacce utente, l'Observer è vitale per garantire che la UI rifletta sempre lo stato attuale dei dati senza dover implementare costosi meccanismi di polling. Permette di creare interfacce dinamiche e reattive in cui i componenti si limitano ad "ascoltare" le modifiche pertinenti.

6.5.3 Utilizzo del pattern nel progetto

Applicato nel frontend Next.js per la gestione dello stato globale al Livello 2. I Context Provider (es. `SessionContext`, `CartContext`) mantengono lo stato condiviso; i componenti UI che consumano tali contesti tramite hook fungono da Observer. Nel backend, il componente `SSEBroadcaster` emette eventi Server-Sent Events (SSE) verso i client connessi: aggiornamenti di chat, modifiche al carrello e variazioni di stato dei ticket. I componenti frontend ricevono questi eventi tramite `EventSource` e propagano le notifiche ai contesti, innescando il re-render selettivo delle viste interessate.

7 Database

Questa sezione descrive l'architettura dei dati del sistema *SmartOrder*, definendo il ruolo del database, le macro-aree informative gestite, i vincoli di integrità applicati e le modalità di interscambio dati con il sistema gestionale esterno.

7.1 Ruolo del database nel sistema

Il sistema si avvale di **PostgreSQL** (in hosting cloud su Railway) come unica fonte dati transazionale (Single Source of Truth) per il backend FastAPI. Il database relazionale assolve a tre ruoli fondamentali:

- **Persistenza Transazionale:** memorizza in modo consistente lo stato delle sessioni, dei carrelli in lavorazione, dei ticket di assistenza e lo storico degli ordini consolidati.
- **Replica del Catalogo ERP:** ospita le tabelle anagrafiche dei prodotti e dei clienti sincronizzate dal gestionale aziendale, permettendo al sistema di operare con logiche di business locali (es. verifica assortimenti).
- **Motore di Ricerca Vettoriale:** tramite l'estensione `pgvector`, il database archivia gli *embeddings* generati dai modelli LLM e risolve le query semantiche di "Vector Search", mappando le descrizioni in linguaggio naturale ai codici articolo reali.

7.2 Macro-aree informative

Il modello relazionale del database è logicamente suddiviso in macro-aree, ciascuna responsabile di un dominio specifico dell'applicazione.

Area Logica	Contenuto principale e Responsabilità
Identità e Accesso	Entità relative agli utenti, ruoli (Cliente, Operatore), credenziali crittografate (hash scrypt) e gestione dei token/sessioni attive.
Catalogo e Assortimento	Anagrafica articoli (`anaart`), metadati descrittivi, *embeddings* vettoriali per la ricerca semantica e tabelle di visibilità/assortimento per singolo cliente (`asscli`).
Carrello e Ordini	Record transazionali che rappresentano il carrello corrente della sessione, le singole righe prodotto con le relative quantità e lo storico degli ordini finalizzati pronti per la consultazione (Storico Ordini).
Ticketing e Chat	Entità per la gestione dell'assistenza (*Human-in-the-loop*). Include i record dei Ticket (stati: "Aperto", "In lavorazione", "Chiuso"), l'assegnazione all'Operatore (Lock) e il tracciato cronologico (Log) dei messaggi di chat scambiati.
Log AI e Ottimizzazione	Archiviazione diagnostica delle interazioni con il modello LLM. Comprende il *Confidence Score*, i feedback degli utenti e le discrepanze generate dalle correzioni manuali ai carrelli, utili al *retraining*.

Tabella 7: Aree logiche del modello dati relazionale.

7.3 Struttura delle principali tabelle

In base allo schema SQL in uso, di seguito è riportata la struttura e il significato dei campi più rilevanti per le entità centrali del sistema.

7.3.0.1 Anagrafica Articoli (`anaart`) Tabella fondamentale per la gestione del catalogo ordini:

- `cod_art`: codice articolo (Primary Key).
- `des_art`: descrizione estesa del prodotto.
- `stato`: stato logico di disponibilità del prodotto.
- `linea`: indica il settore reale a cui appartiene il prodotto (es. acqua, vino, ecc.).
- `famiglia`: indica la fascia o qualità del prodotto (es. premium, standard, ecc.).
- `des_um`: unità di misura base di vendita (es. casse, cartone).
- `des_tipo_um`: unità di misura dei singoli prodotti all'interno della singola confezione (es. pezzi, bottiglie).
- `pezzi_conf`: quantità calcolata in `des_tipo_um` situata all'interno della misura base `des_um`.
- `peso_netto_conf`: peso netto per singola unità di `des_um`.

Nota operativa: L'attributo nativo `settore` presente a schema non contiene in realtà i valori di business indicanti il settore del prodotto, i quali si trovano invece nel campo `linea`.

7.3.0.2 Sessioni e Chat (`chat_sessions, chat_messages`) Il nucleo conversazionale salva lo stato delle interazioni in maniera persistente:

- `status`: indica se la sessione è *active*, *completed* o *archived*.
- `sender` (`chat_messages`): traccia l'emittente esatto del messaggio tra *user*, *ai*, *operator* e *system*.
- `image_data` e `ocr_text`: permettono l'invio di ordini multimodali memorizzando un'immagine decodificata in base64 e l'interpretazione testuale del servizio LLM associato.

7.3.0.3 Carrello (`cart_items`) Le righe del carrello in elaborazione prevedono una cronologia di modifica e confidenza:

- `cod_art` e `qta`: riferimento all'articolo in assortimento e alle quantità richieste dall'utente.
- `source` e `last_updated_by`: attribuiscono l'inserimento/modifica iniziale e finale a un utente, all'intelligenza artificiale o a un operatore.
- `ai_confidence`: livello di certezza estrapolato dal LLM (fondamentale per far scattare controlli automatizzati o trigger per il backoffice in caso di bassi valori).

7.3.0.4 Ordini Finalizzati (`orders, order_items`) Rappresentano la formalizzazione del carrello al netto delle modifiche di *Human-in-the-Loop*. Gli ordini mantengono un tracciamento dei campi `source` per finalità di audit. Troviamo inoltre il flag di stato `esportato` per gestire l'avvenuta sincronizzazione JSON.

7.3.0.5 Identità Temporanee e Persistenti (`app_users`, `anacli`, `asscli`)

- `app_users`: include il ruolo autenticato (*customer* o *admin*) e la cartella di esportazione.
- `anacli`: generalità e ragione sociale del cliente.
- `asscli`: tabella ponte di assortimento (collega `cod_cli` e `cod_art`). Esclusivamente quanto è qui presente risulta ordinabile dal Cliente interessato.

7.4 Vincoli dati rilevanti

Per garantire la robustezza del sistema e prevenire l'invio di ordini commercialmente invalidi al sistema ERP, il database impone rigorosi vincoli di integrità e di dominio:

- **Vincolo di Assortimento (Foreign Key logica):** un prodotto può essere inserito nel carrello di una sessione solo se esiste una corrispondenza valida nella tabella degli assortimenti autorizzati (`asscli`) per il Cliente in esame.
- **Vincolo di Disponibilità:** controlli a livello database (`anaart`) impediscono l'immissione in ordine di articoli marcati con stato "sospeso" o "non disponibile".
- **Integrità Referenziale Standard:** chiavi esterne (Foreign Keys) con regole di propagazione *ON DELETE CASCADE/RESTRICT* collegano indissolubilmente le righe dell'ordine all'ID Ordine padre e alla relativa anagrafica Cliente, prevenendo record orfani.
- **Vincolo di Quantità Commerciale:** i campi quantitativi delle righe ordine possiedono vincoli (*CHECK constraints*) per impedire valori nulli, negativi o incompatibili con i multipli di vendita (es. logica a "casse").

7.5 Output verso ERP

Il database di SmartOrder funge da ambiente di *staging* e composizione. Al termine del processo di checkout, i dati transazionali (intestazione ordine, righe, quantità e ID cliente) estratti dalle tabelle relazionali subiscono un processo di serializzazione.

Il sistema genera un **payload strutturato in formato JSON** conforme al contratto di integrazione concordato con il fornitore del gestionale. Questo file non viene immagazzinato a lungo termine nel database cloud, ma viene immediatamente depositato nel **File Storage condiviso (Bucket S3)**, che rappresenta il reale punto di aggancio e consegna formale della transazione verso il sistema ERP aziendale.

8 Architettura di dettaglio

In questa sezione vengono descritti gli elementi di dettaglio architetturale del sistema, suddivisi per ambito backend e frontend. Per ciascun componente/classe/interfaccia sono riportati:

- Titolo dell'elemento;
- Descrizione della responsabilita;
- Descrizione di attributi e metodi;
- Diagramma UML dedicato (generato in PlantUML ed esportato in PNG).

8.1 Backend - C4 Livello 4: Classi e interfacce

La sottosezione presenta classi e interfacce backend identificate nel C4 Livello 4.

8.1.1 Layer 1 - Controllers (Inbound Ports)

8.1.1.1 AuthController **Descrizione della classe:** gestisce gli endpoint di autenticazione e cambio password, traducendo le richieste HTTP in invocazioni ai casi d'uso applicativi.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `login(credentials: CredentialsDTO)`: avvia autenticazione utente e genera la sessione applicativa.
- `logout(authsessionId: String)`: invalida la sessione autenticata corrente.
- `changePassword(req: PasswordChangeDTO)`: gestisce il flusso di aggiornamento credenziali.

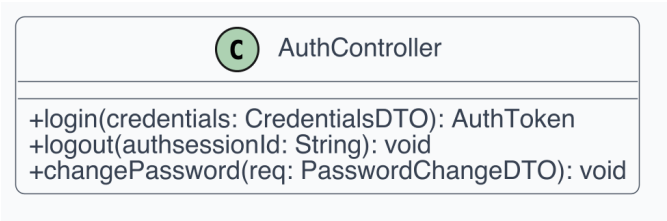

Figura 4: UML della classe `AuthController`.

8.1.1.2 ConversationController **Descrizione della classe:** espone gli endpoint per la conversazione multimodale e il recupero dello storico chat.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `processInput(req: MultimodalInputDTO)`: riceve input testuale/audio e avvia l'elaborazione conversazionale.
- `resetSession(chatsessionId: String)`: azzera il contesto della sessione chat.
- `getChatHistory(chatsessionId: String)`: recupera la cronologia dei messaggi della sessione.

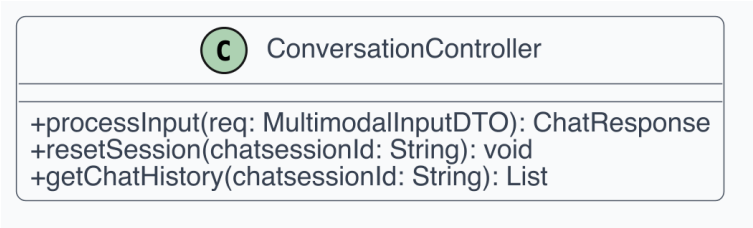

Figura 5: UML della classe `ConversationController`.

8.1.1.3 CartController **Descrizione della classe:** governa le operazioni sul carrello (inserimento, modifica, rimozione) e il checkout finale.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `addProduct(req: CartActionDTO)`: inserisce una riga prodotto nel carrello di sessione.
- `updateQuantity(req: CartActionDTO)`: aggiorna la quantita della riga selezionata.
- `removeProduct(req: CartActionDTO)`: elimina un prodotto dal carrello.
- `checkout(chatsessionId: String)`: finalizza il carrello e avvia la generazione ordine.

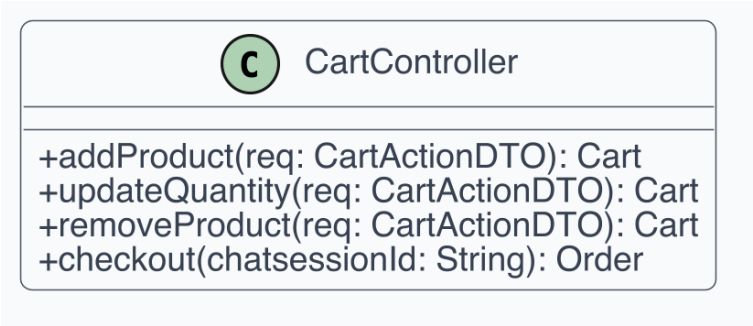

Figura 6: UML della classe `CartController`.

8.1.1.4 OrderController **Descrizione della classe:** espone le API per consultazione storico ordini e dettaglio singolo ordine.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `getOrders(clienteId: String)`: recupera la lista ordini del cliente.
- `getOrderDetails(orderId: String)`: recupera i dettagli completi dell'ordine richiesto.

Figura 7: UML della classe `OrderController`.

8.1.1.5 TicketController

Descrizione della classe: gestisce l'apertura, la presa in carico e la chiusura dei ticket di assistenza Human-in-the-Loop.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `openTicket(req: TicketRequestDTO)`: crea un nuovo ticket applicativo.
- `takeCharge(ticketId: String, operatorId: String)`: assegna il ticket a un operatore.
- `closeTicket(ticketId: String)`: marca il ticket come risolto.
- `getOpenTickets()`: restituisce i ticket ancora aperti.

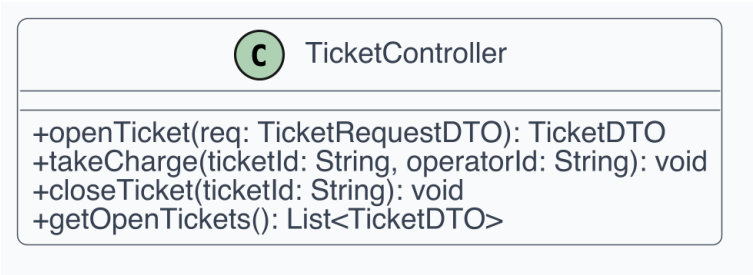

Figura 8: UML della classe `TicketController`.

8.1.1.6 FeedbackController

Descrizione della classe: espone le API di raccolta feedback e verifica dei log AI per monitoraggio e miglioramento continuo.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `submitFeedback(req: FeedbackDTO)`: registra feedback utente sul comportamento AI.
- `getLogs(filter: LogFilterDTO)`: recupera i log applicando filtri operativi.
- `validateLog(logId: String, status: ValidStatus)`: marca l'esito di validazione di un log.

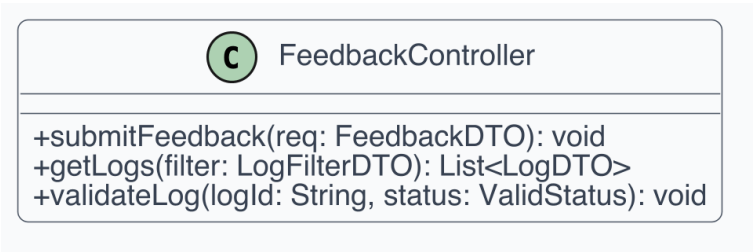

Figura 9: UML della classe `FeedbackController`.

8.1.1.7 ClientController **Descrizione della classe:** gestisce il recupero delle informazioni specifiche del cliente connesso, integrandole eventualmente da sistemi esterni.

Figura 10: UML della classe `ClientController`.

8.1.1.8 HealthController **Descrizione della classe:** fornisce un endpoint per l'health check e il monitoraggio dello stato di salute del backend.

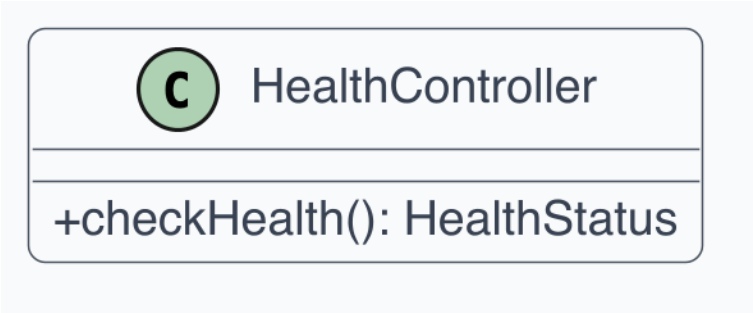

Figura 11: UML della classe `HealthController`.

8.1.1.9 ProductController **Descrizione della classe:** espone le API di ricerca ed esplorazione del catalogo prodotti, filtrato per assortimento cliente.

Figura 12: UML della classe `ProductController`.

8.1.1.10 RegisterController **Descrizione della classe:** gestisce la registrazione e il provisioning di nuovi clienti e account interni (secret-register).

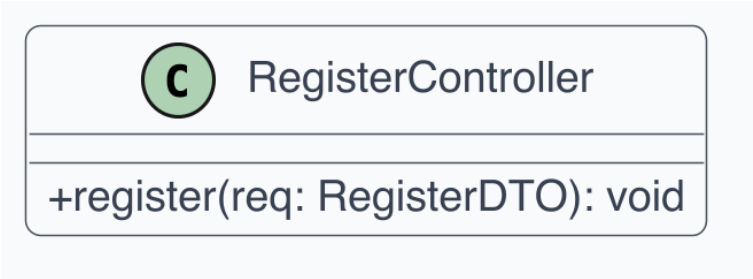

Figura 13: UML della classe `RegisterController`.

8.1.1.11 SSEController **Descrizione della classe:** gestisce la connessione Server-Sent Events rimpiazzando in parte il vecchio canale WebSocket per le notifiche in streaming.

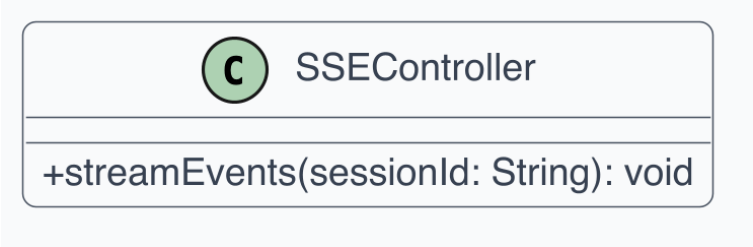

Figura 14: UML della classe `SSEController`.

8.1.2 Layer 2 - Application Services (Use Cases)

8.1.2.1 AuthService **Descrizione della classe:** incapsula i casi d'uso di autenticazione e invalidazione sessione.

Descrizione degli attributi: non sono esplicitati attributi nel diagramma; il servizio e modellato come componente applicativo stateless.

Descrizione dei metodi:

- `authenticateUser(email: String, pass: String)`: valida credenziali e produce token di autenticazione.
- `invalidateSession(authsessionId: String)`: disattiva una sessione autenticata.

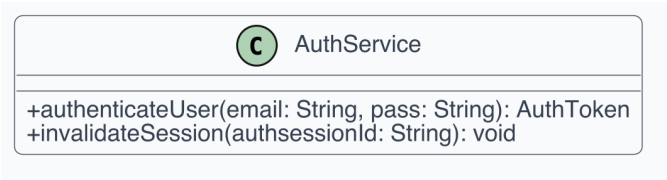

Figura 15: UML della classe `AuthService`.

8.1.2.2 ConversationService **Descrizione della classe:** implementa il flusso conversazionale (testo e audio) e la gestione del contesto dialogico.

Descrizione degli attributi: non sono esplicitati attributi di stato nel diagramma.

Descrizione dei metodi:

- `handleText(text: String, sessionId: String)`: processa input testuale e restituisce risposta chat.
- `handleAudio(audioBlob: File, sessionId: String)`: gestisce input audio con trascrizione e interpretazione.
- `clearContext(chatsessionId: String)`: cancella lo stato conversazionale associato alla sessione.

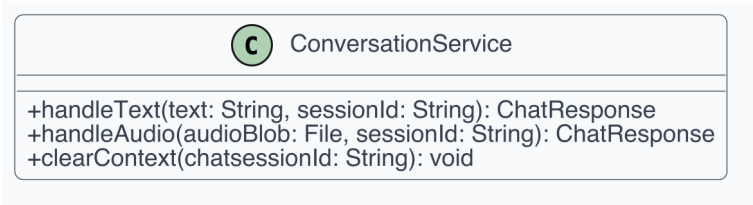

Figura 16: UML della classe `ConversationService`.

8.1.2.3 CartService

Descrizione della classe: concentra la logica di business del carrello e la finalizzazione ordine.

Descrizione degli attributi: non sono indicati attributi persistenti nel diagramma.

Descrizione dei metodi:

- `manipulateCart(action: CartAction)`: applica operazioni di aggiunta/modifica/rimozione su carrello.
- `finalizeOrder(chatsessionId: String)`: converte il carrello in ordine definitivo.
- `validateAssortment(clienteId: String, prodottoId: String)`: verifica interna di validita assortimento cliente-articolo.

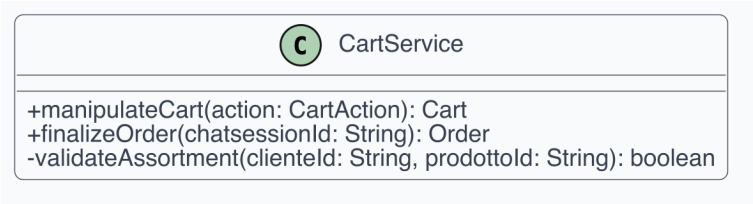

Figura 17: UML della classe `CartService`.

8.1.2.4 OrderService

Descrizione della classe: fornisce i casi d'uso di consultazione storico ordini e dettaglio ordine.

Descrizione degli attributi: nessun attributo esplicitato nel diagramma.

Descrizione dei metodi:

- `getOrdersByCliente(clienteId: String)`: recupera gli ordini di un cliente.
- `getOrderDetail(orderId: String)`: restituisce il dettaglio di un ordine specifico.

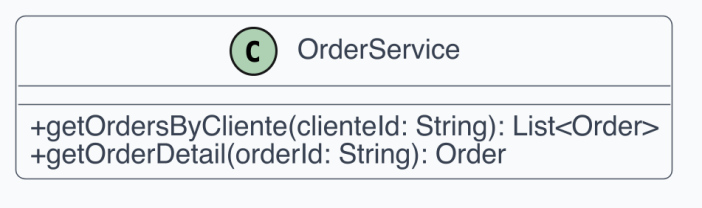

Figura 18: UML della classe `OrderService`.

8.1.2.5 VectorSearchService **Descrizione della classe:** gestisce embedding e ricerca semantica dei prodotti nel catalogo cliente.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma.

Descrizione dei metodi:

- `generateEmbedding(testo: String)`: produce il vettore semantico del testo in input.
- `semanticSearch(vector: Vector, clienteId: String)`: esegue ricerca vettoriale filtrata per cliente.

Figura 19: UML della classe `VectorSearchService`.

8.1.2.6 TicketService **Descrizione della classe:** implementa l'intero ciclo di vita del ticket di assistenza.

Descrizione degli attributi: nessun attributo esplicitato nel diagramma.

Descrizione dei metodi:

- `createTicket(clienteId: String, reason: String)`: crea un ticket a partire da richiesta cliente.
- `assignOperator(ticketId: String, operatorId: String)`: assegna un operatore al ticket.
- `resolveTicket(ticketId: String)`: chiude il ticket come risolto.

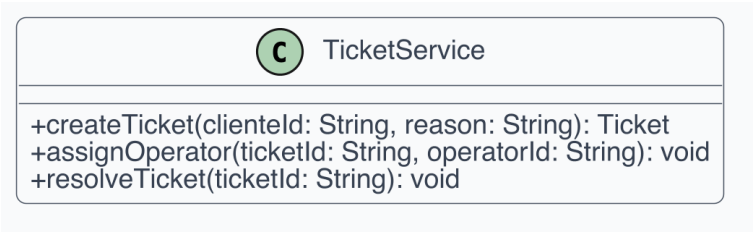

Figura 20: UML della classe `TicketService`.

8.1.2.7 FeedbackService **Descrizione della classe:** presidia logging delle correzioni manuali e raccolta feedback negativi verso il modulo AI.

Descrizione degli attributi: il diagramma non esplicita attributi interni.

Descrizione dei metodi:

- `logCorrection(delta: String, chatsessionId: String)`: persiste lo scostamento tra proposta AI e correzione utente.
- `saveNegativeFeedback(feedback: FeedbackData)`: salva feedback negativo per analisi qualitativa.

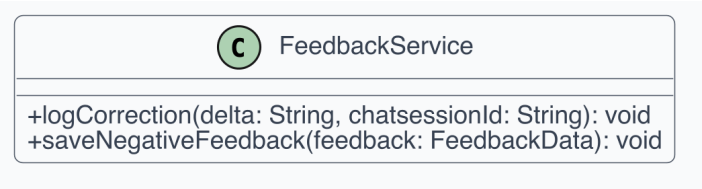

Figura 21: UML della classe `FeedbackService`.

8.1.2.8 ClientService **Descrizione della classe:** implementa la logica di business di recupero e verifica dei dati anagrafici del cliente registrato.

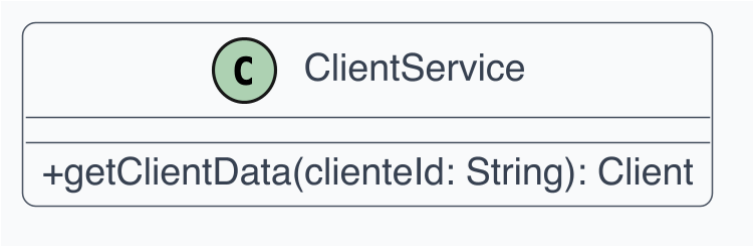

Figura 22: UML della classe `ClientService`.

8.1.2.9 ProductService **Descrizione della classe:** incapsula la logica per il filtraggio e l'accesso ottimizzato all'assortimento e al catalogo articoli.

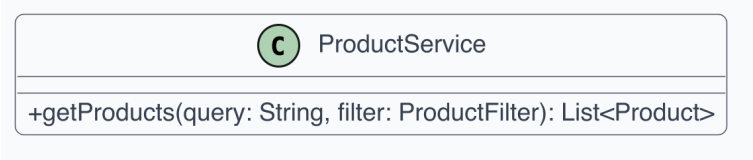

Figura 23: UML della classe `ProductService`.

8.1.2.10 SSEBroadcaster **Descrizione della classe:** si occupa del dispatch e della propagazione degli eventi realtime a tutte le sessioni connesse, disaccoppiando l'infrastruttura di stream dai domini.

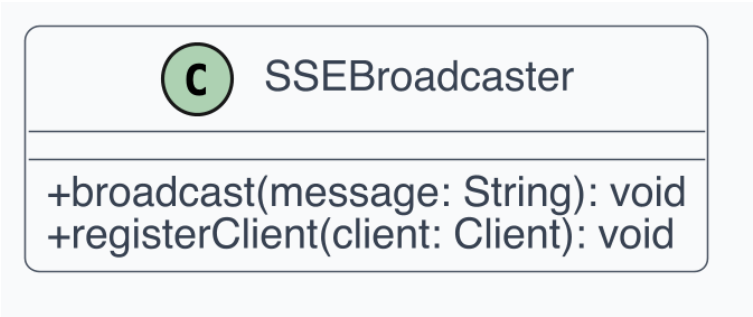

Figura 24: UML della classe `SSEBroadcaster`.

8.1.3 Layer 3 - Core Domain (Entities & Value Objects)

8.1.3.1 Cliente **Descrizione della classe:** rappresenta il cliente commerciale autenticato e abilitato all'inserimento ordini.

Descrizione degli attributi:

- `id`: identificativo univoco del cliente.
- `ragioneSociale`: denominazione legale dell'azienda cliente.
- `email`: contatto email principale per autenticazione/comunicazioni.

Descrizione dei metodi: non sono definiti metodi nel diagramma per questa entita.

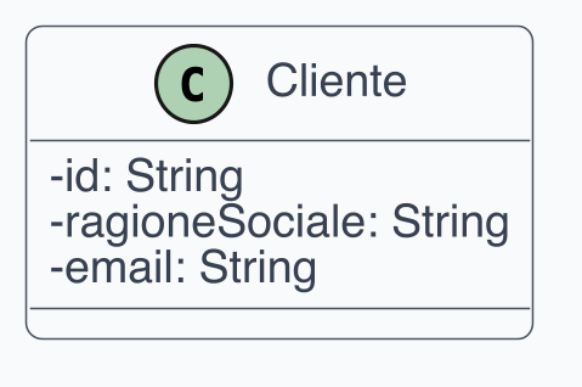

Figura 25: UML della classe `Cliente`.

8.1.3.2 Operatore **Descrizione della classe:** identifica l'utente interno che prende in carico ticket e supervisiona i casi critici.

Descrizione degli attributi:

- `id`: identificativo univoco operatore.
- `nome`: nominativo operatore.

Descrizione dei metodi: non sono definiti metodi nel diagramma.

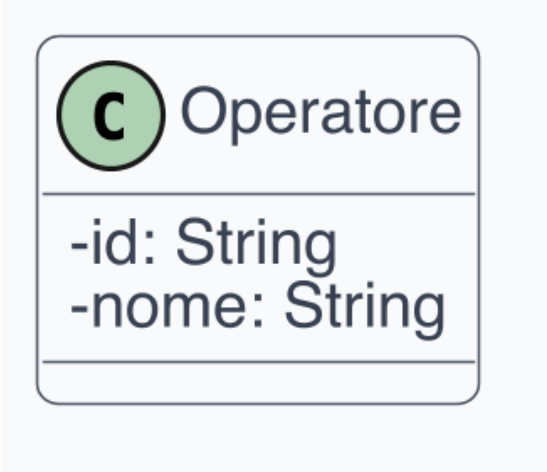

Figura 26: UML della classe `Operatore`.

8.1.3.3 Order **Descrizione della classe:** rappresenta l'ordine consolidato pronto per storicizzazione ed esportazione verso ERP.

Descrizione degli attributi:

- `id`: identificativo ordine.
- `clienteId`: riferimento al cliente proprietario dell'ordine.
- `dataOrdine`: timestamp di emissione.
- `righe`: collezione delle righe ordine derivate dal carrello.

Descrizione dei metodi: nel diagramma non sono mostrati metodi per questa entita.

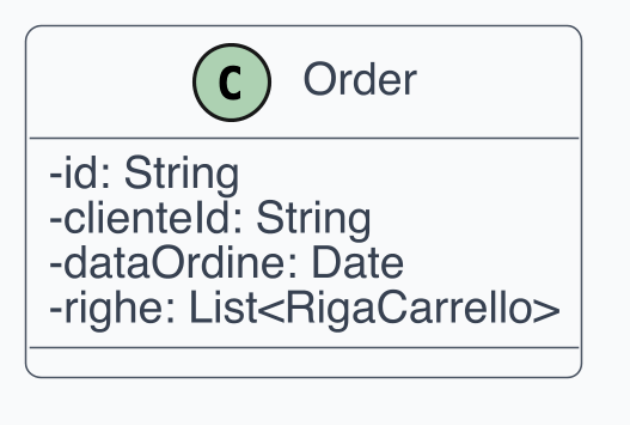

Figura 27: UML della classe `Order`.

8.1.3.4 Carrello **Descrizione della classe:** modella lo stato temporaneo dell'ordine in costruzione per una sessione chat.

Descrizione degli attributi:

- `chatsessionId`: identificativo sessione conversazionale associata.
- `clienteId`: cliente a cui il carrello appartiene.
- `righe`: elenco righe prodotto correnti.

Descrizione dei metodi:

- `calcolaTotalePezzi()`: restituisce la somma totale delle quantita nel carrello.

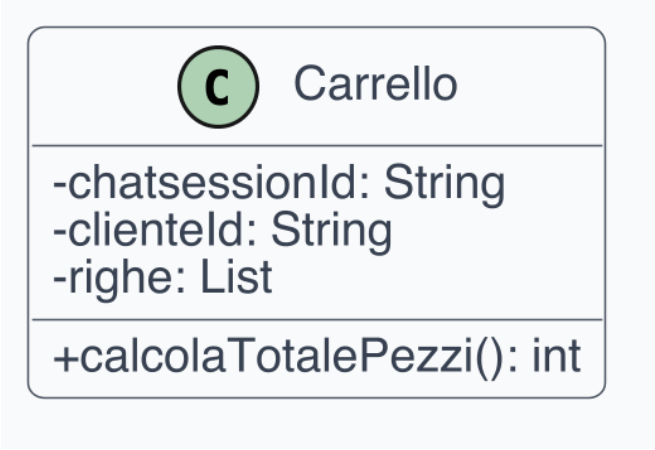

Figura 28: UML della classe `Carrello`.

8.1.3.5 RigaCarrello **Descrizione della classe:** rappresenta una singola posizione prodotto-quantita nel carrello.

Descrizione degli attributi:

- `prodottoId`: riferimento all'articolo di catalogo.
- `quantita`: numero unita richieste.
- `confidenceScore`: affidabilita della proposta AI su quella riga.

Descrizione dei metodi: non sono definiti metodi nel diagramma.

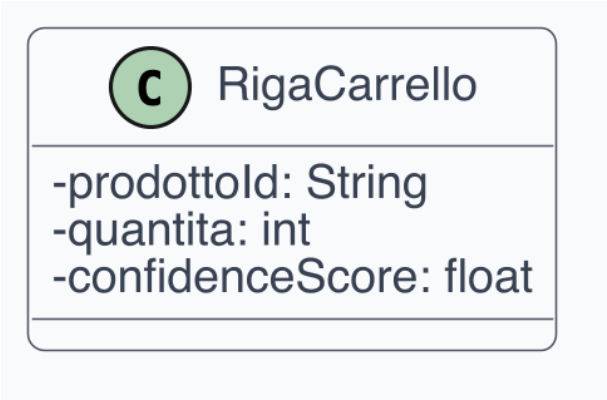

Figura 29: UML della classe `RigaCarrello`.

8.1.3.6 Session **Descrizione della classe:** mantiene il contesto di conversazione tra cliente e sistema durante la composizione ordine.

Descrizione degli attributi:

- `id`: identificativo sessione.
- `clientId`: identificativo del cliente associato.
- `status`: stato della sessione (aperta, chiusa, sospesa).
- `messages`: storico messaggi scambiati.

Descrizione dei metodi: non sono presenti metodi nel diagramma.

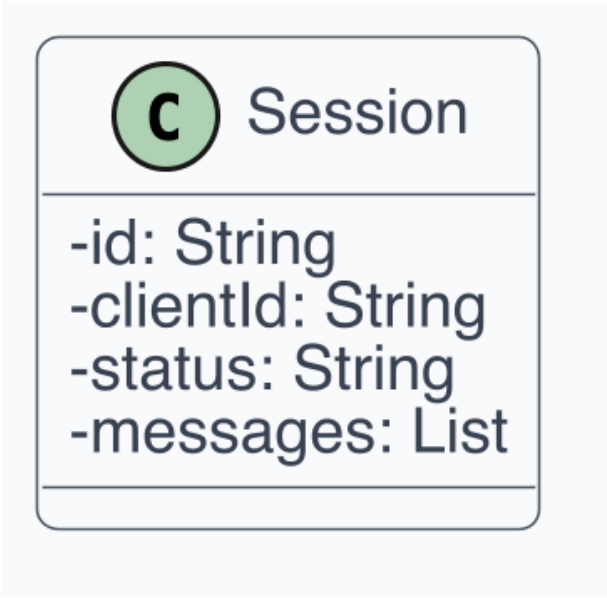

Figura 30: UML della classe `Session`.

8.1.3.7 Ticket **Descrizione della classe:** rappresenta una richiesta di supporto aperta per intervento umano.

Descrizione degli attributi:

- `id`: identificativo ticket.
- `clienteId`: cliente che ha aperto il ticket.
- `stato`: stato corrente del ticket.
- `dataApertura`: data e ora di apertura.

Descrizione dei metodi: il diagramma non esplicita metodi per questa entita.

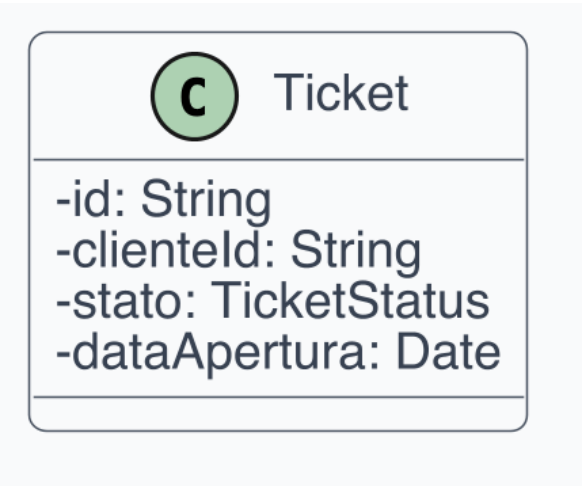

Figura 31: UML della classe `Ticket`.

8.1.3.8 LogAI **Descrizione della classe:** traccia input, output e correzioni legate alle elaborazioni AI per audit e miglioramento modello.

Descrizione degli attributi:

- `id`: identificativo del log.
- `inputOriginario`: testo/audio trascritto ricevuto dal cliente.
- `rispostaLLM`: proposta generata dal modello linguistico.
- `correzioneDelta`: differenza applicata manualmente da utente/operatore.
- `stato`: stato del log nel workflow di validazione.

Descrizione dei metodi: non sono definiti metodi nel diagramma.

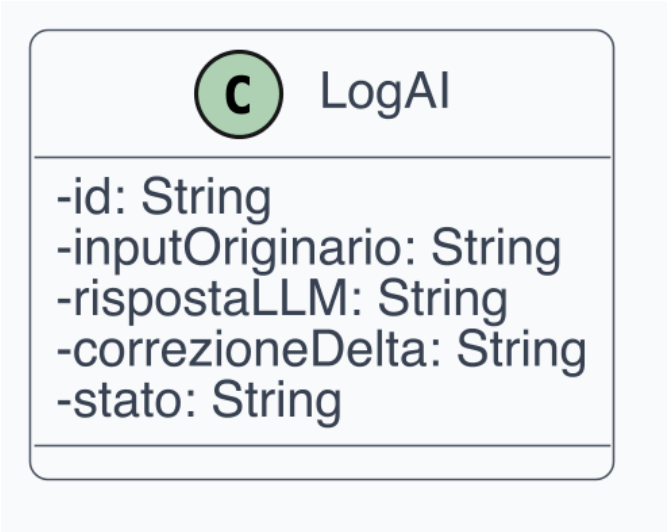

Figura 32: UML della classe `LogAI`.

8.1.4 Layer 4 - Outbound Ports (Interfaces)

8.1.4.1 IUserRepository **Descrizione della classe/interfaccia:** definisce il contratto di accesso ai dati utente e alle regole di assortimento.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `findByEmail(email: String)`: recupera il cliente associato all'email.
- `getAssortment(clienteId: String)`: restituisce i codici prodotto autorizzati per cliente.

Figura 33: UML dell'interfaccia `IUserRepository`.

8.1.4.2 ISessionManager

Descrizione della classe/interfaccia: astrae la persistenza di sessioni e carrelli applicativi.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `saveSession(session: SessionData)`: persiste lo stato di sessione.
- `getCart(chatsessionId: String)`: recupera il carrello della sessione.
- `saveCart(cart: Carrello)`: aggiorna il carrello persistito.

Figura 34: UML dell'interfaccia `ISessionManager`.

8.1.4.3 IOrderRepository

Descrizione della classe/interfaccia: definisce le operazioni di persistenza e ricerca ordini.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `saveOrder(order: Order)`: salva l'ordine finalizzato.
- `findByCliente(clienteId: String)`: recupera ordini del cliente.
- `findById(orderId: String)`: recupera ordine per identificativo.
- `getOrders(filters, page, pageSize)`: ricerca ordini con filtri e paginazione.

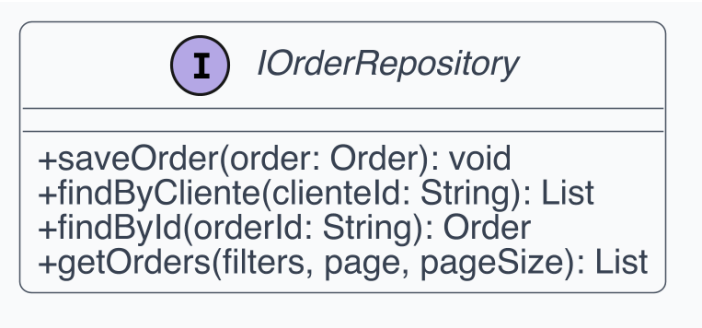

Figura 35: UML dell'interfaccia `IOrderRepository`.

8.1.4.4 ITicketRepository

Descrizione della classe/interfaccia: astrae persistenza e query dei ticket di assistenza.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `save(ticket: Ticket)`: persiste ticket nuovo o aggiornato.

- `findAllOpen()`: recupera i ticket in stato aperto.
- `getTickets(filters)`: ricerca ticket con criteri dinamici.

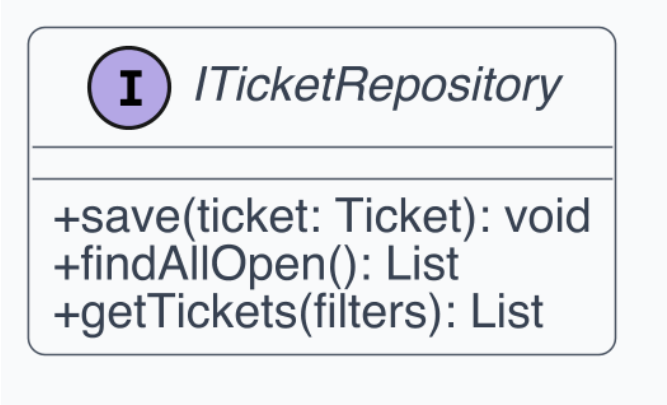

Figura 36: UML dell'interfaccia `ITicketRepository`.

8.1.4.5 ILogAIRepository

Descrizione della classe/interfaccia: definisce l'accesso ai dati di logging AI.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `saveLog(log: LogAI)`: salva un evento di log AI.
- `findPendingLogs()`: recupera i log in attesa di validazione.

Figura 37: UML dell'interfaccia `ILogAIRepository`.

8.1.4.6 IFileExporter

Descrizione della classe/interfaccia: espone il contratto di esportazione file verso sistemi esterni.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `writeOrderAsJSON(order: Order)`: serializza ordine in formato JSON di integrazione.
- `writeOrdersAsCSV(clientId: String)`: genera export CSV per storico/reportistica.

Figura 38: UML dell'interfaccia `IFileExporter`.

8.1.4.7 IAIClient

Descrizione della classe/interfaccia: formalizza il contratto verso i servizi di AI generativa e speech-to-text.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `transcribeAudio(audio: File)`: converte audio in testo.
- `interpretIntent(text: String, context: String)`: classifica intenti e produce decisione AI.
- `createEmbedding(text: String)`: calcola embedding vettoriale del testo.

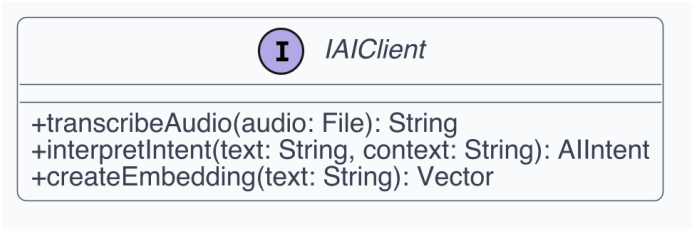

Figura 39: UML dell'interfaccia `IAIClient`.

8.1.4.8 INotificationPort

Descrizione della classe/interfaccia: definisce notifiche realtime verso operatori e client attivi.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `notifyOperator(operatorId: String, ticket: Ticket)`: invia notifica di presa in carico/aggiornamento ticket.
- `sendRealtimeUpdate(chatsessionId: String, payload: String)`: invia aggiornamento realtime su sessione chat.

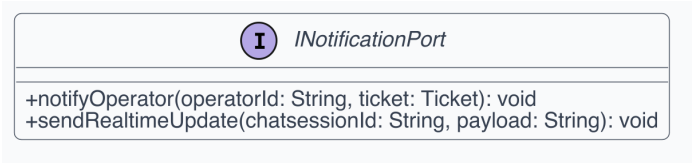

Figura 40: UML dell'interfaccia `INotificationPort`.

8.1.5 Layer 5 - Infrastructure Adapters

8.1.5.1 PostgresAdapter **Descrizione della classe:** implementa le porte repository interagendo con PostgreSQL e traducendo query applicative in operazioni SQL.

Descrizione degli attributi:

- `connectionString`: stringa di connessione al database.

Descrizione dei metodi:

- `executeSQL(query: String)`: esegue query verso DB e restituisce il risultato elaborabile dagli adapter superiori.

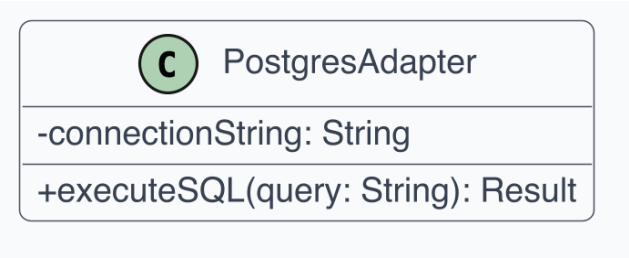

Figura 41: UML della classe `PostgresAdapter`.

8.1.5.2 OpenAIAdapter **Descrizione della classe:** realizza il contratto `IAIClient` mappando chiamate di dominio verso le API OpenAI.

Descrizione degli attributi:

- `apiKey`: chiave di autenticazione verso il servizio OpenAI.

Descrizione dei metodi:

- `transcribeAudio(audio: File)`: invoca il modello speech-to-text.
- `interpretIntent(text: String, context: String)`: invia prompt contestualizzato al modello conversazionale.
- `createEmbedding(text: String)`: richiede embedding per vector search.

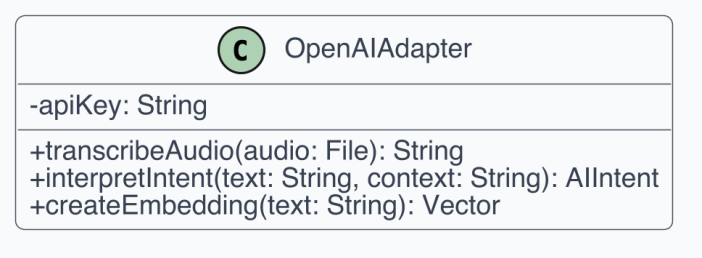

Figura 42: UML della classe `OpenAIAdapter`.

8.1.5.3 FileSystemAdapter **Descrizione della classe:** implementa la porta di esportazione ordini scrivendo file nel canale condiviso con ERP.

Descrizione degli attributi:

- `sharedFolderPath`: percorso logico/fisico della cartella di integrazione.

Descrizione dei metodi:

- `writeOrderAsJSON(order: Order)`: serializza e salva l'ordine in JSON.

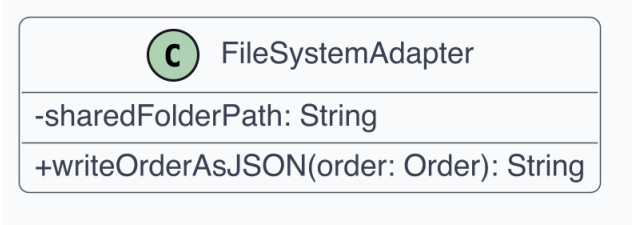

Figura 43: UML della classe `FileSystemAdapter`.

8.1.5.4 WebSocketAdapter **Descrizione della classe:** concretizza la porta di notifica realtime tramite canale WebSocket.

Descrizione degli attributi: il diagramma non esplicita attributi persistenti per questa classe.

Descrizione dei metodi:

- `notifyOperator(operatorId: String, ticket: Ticket)`: invia notifiche operative ad un operatore specifico.
- `sendRealtimeUpdate(chatsessionId: String, payload: String)`: propaga aggiornamenti in tempo reale alla sessione client.

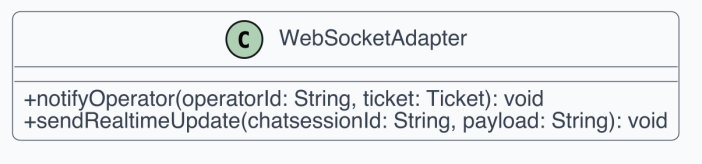

Figura 44: UML della classe `WebSocketAdapter`.

8.2 Frontend - C4 Livello 4

La sottosezione presenta i componenti frontend identificati nel C4 Livello 4.

8.2.1 Layer 1 - Routing & Page Layer (Next.js App Router)

8.2.1.1 RouteGuard **Descrizione del componente:** presidia la sicurezza di navigazione verificando validita della sessione e autorizzazioni per ruolo prima dell'accesso alle route protette.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (logica middleware stateless).

Descrizione dei metodi:

- `validateSession(token: JWT)`: verifica la validita del token della sessione corrente.
- `checkRole(requiredRole: UserRole)`: applica controllo di autorizzazione per ruolo utente.
- `redirectToLogin()`: effettua redirect verso la pagina di accesso in caso di sessione non valida.

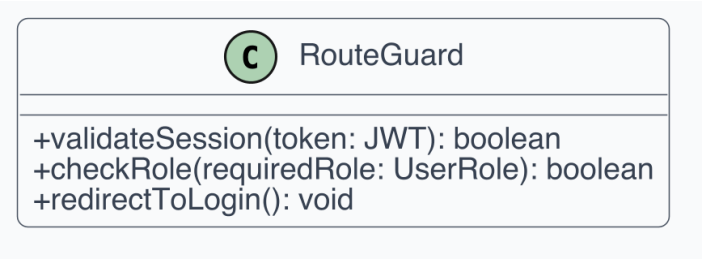

Figura 45: UML del componente `RouteGuard`.

8.2.1.2 AuthPages **Descrizione del componente:** raggruppa le pagine di autenticazione (login e recupero password) e gestisce l'invio delle credenziali al backend.

Descrizione degli attributi: il diagramma non esplicita attributi persistenti.

Descrizione dei metodi:

- `renderLoginForm()`: renderizza il form di accesso utente.
- `renderForgotPasswordForm()`: renderizza il form di recupero password.
- `submitCredentials(credentials: CredentialsDTO)`: inoltra le credenziali all'API di autenticazione.

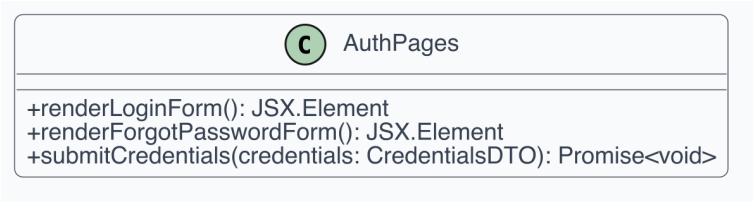

Figura 46: UML del componente `AuthPages`.

8.2.1.3 ClientePages **Descrizione del componente:** incapsula le viste principali dell'area cliente (chat, carrello, storico, profilo) con approccio mobile-first come da vincoli C4L4F.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma.

Descrizione dei metodi:

- `openChat()`: apre la vista conversazionale cliente-AI.
- `openCart()`: apre la vista di composizione carrello.
- `openOrderHistory()`: apre la vista storico ordini cliente.
- `openProfile()`: apre la vista profilo e dati account.

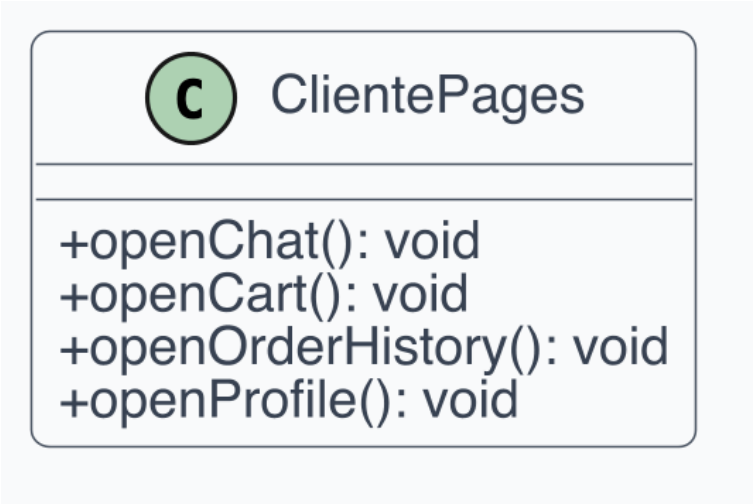

Figura 47: UML del componente `ClientePages`.

8.2.1.4 BackofficePage

Descrizione del componente: realizza la rotta unica desktop-first per operatore e centralizza la navigazione a tab (ticket, ordini globali, AI log).

Descrizione degli attributi:

- `tabs`: collezione tab disponibili in dashboard backoffice.
- `activeTab`: tab correntemente attiva nella vista operatore.

Descrizione dei metodi:

- `switchTab(tab: BackofficeTab)`: commuta la sezione operativa visualizzata.
- `renderTicketTable()`: renderizza la tabella ticket aperti/in lavorazione.
- `renderOrdersTable()`: renderizza la tabella ordini globali.
- `renderAILogTable()`: renderizza la tabella delle segnalazioni AI.

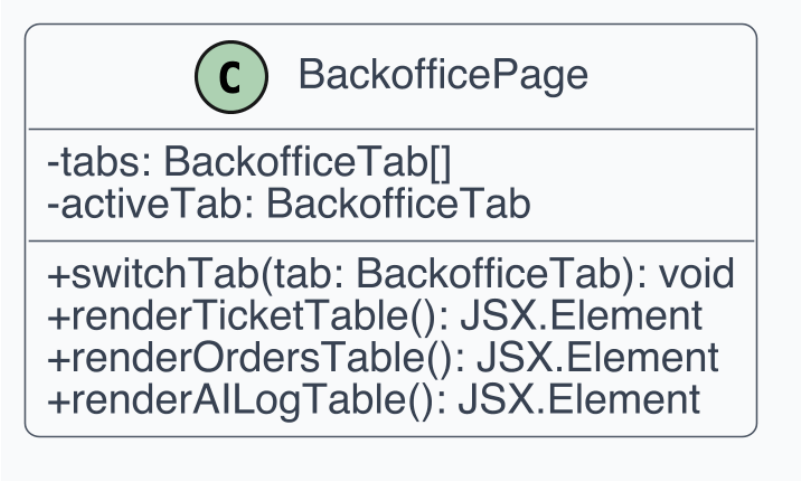

Figura 48: UML del componente `BackofficePage`.

8.2.1.5 SharedLayout **Descrizione del componente:** fornisce il layout condiviso dell'applicazione (navbar, sidebar e toasts) usato sia dall'area cliente sia dal backoffice.

Descrizione degli attributi: il diagramma non esplicita attributi di stato.

Descrizione dei metodi:

- `renderNavbar()`: costruisce la barra di navigazione principale.
- `renderSidebar()`: costruisce il menu laterale contestuale.
- `showToast(notification: NotificationDTO)`: mostra notifiche utente transitorie.

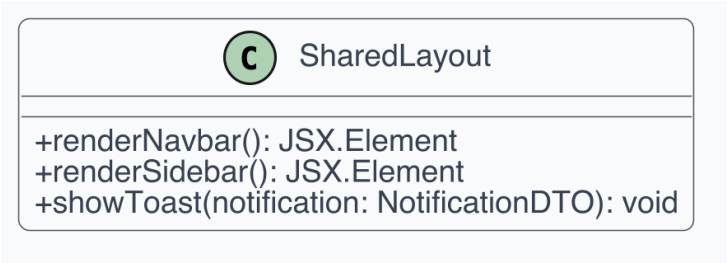

Figura 49: UML del componente `SharedLayout`.

8.2.2 Layer 2 - State Management Layer (React Context + Custom Hooks)

8.2.2.1 AuthContext **Descrizione del componente:** gestisce stato di autenticazione lato client (token, identita e ruolo) e rappresenta la base delle guardie di accesso.

Descrizione degli attributi:

- `token`: token JWT della sessione utente.
- `role`: ruolo applicativo corrente (Cliente o Operatore).
- `userId`: identificativo dell'utente autenticato.

Descrizione dei metodi:

- `login(payload: AuthPayload)`: inizializza lo stato di autenticazione locale.
- `logout()`: azzera lo stato sessione lato client.
- `isAuthenticated()`: verifica se l'utente e autenticato.

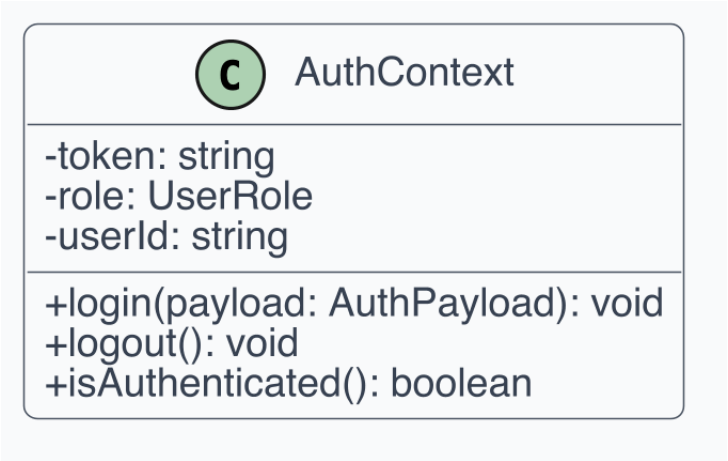

Figura 50: UML del componente `AuthContext`.

8.2.2.2 CartContext **Descrizione del componente:** mantiene lo stato del carrello di sessione, inclusa la confidenza AI esposta in UI operatore.

Descrizione degli attributi:

- `sessionId`: identificativo della sessione chat associata al carrello.
- `items`: righe prodotto attualmente presenti nel carrello.
- `confidenceScore`: metrica di affidabilita AI aggregata/di contesto.

Descrizione dei metodi:

- `upsertItem(item: CartItem)`: aggiunge o aggiorna una riga prodotto nel carrello.
- `removeItem(productId: string)`: rimuove una riga prodotto.
- `clearCart()`: svuota il carrello locale a fine flusso/reset.

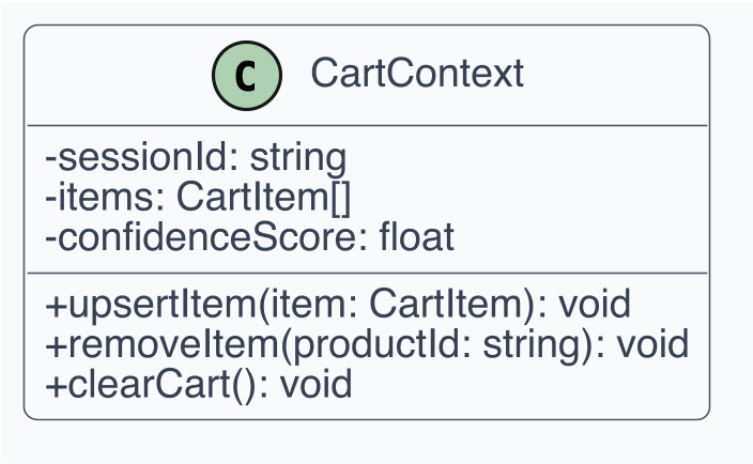

Figura 51: UML del componente `CartContext`.

8.2.2.3 ChatContext **Descrizione del componente:** mantiene lo stato realtime della conversazione (messaggi, connessione, modalita AI/Operatore).

Descrizione degli attributi:

- `messages`: storico messaggi della sessione attiva.
- `connectionState`: stato del canale realtime (connesso/disconnesso/reconnecting).
- `mode`: modalita conversazionale corrente (AI oppure Operatore).

Descrizione dei metodi:

- `appendMessage(message: ChatMessage)`: aggiunge un messaggio al feed corrente.
- `setMode(mode: ChatMode)`: aggiorna la modalita della sessione chat.
- `resetSession()`: azzera il contesto conversazionale lato frontend.

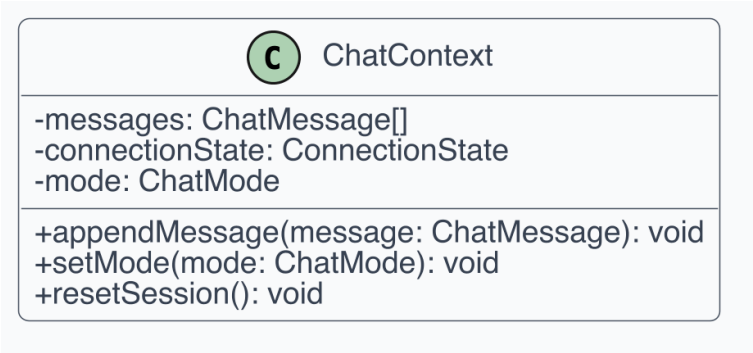

Figura 52: UML del componente `ChatContext`.

8.2.2.4 NotificationContext

Descrizione del componente: centralizza notifiche transitorie e alert applicativi provenienti da azioni utente o eventi realtime.

Descrizione degli attributi:

- `queue`: coda notifiche visualizzabili nella UI.

Descrizione dei metodi:

- `push(notification: NotificationDTO)`: inserisce una nuova notifica in coda.
- `dismiss(notificationId: string)`: rimuove una notifica specifica.
- `clear()`: svuota tutte le notifiche pendenti.

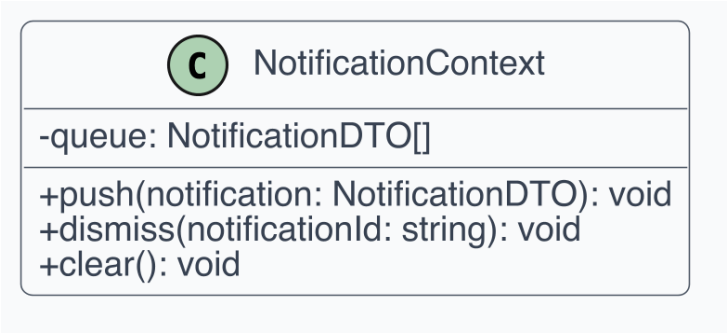

Figura 53: UML del componente `NotificationContext`.

8.2.2.5 CustomHooks

Descrizione del componente: fornisce API di accesso uniforme ai contesti e agli stati applicativi, riducendo duplicazione logica nei componenti UI.

Descrizione degli attributi: non sono esplicitati attributi nel diagramma.

Descrizione dei metodi:

- `useAuth()`: espone stato e operazioni di autenticazione.
- `useCart()`: espone stato e operazioni di carrello.
- `useChat()`: espone stato e operazioni di chat.
- `useTickets()`: espone stato ticket per dashboard operatore.
- `useOrders()`: espone stato ordini per viste storico/globali.

- `useAILogs()`: espone stato AI log per pipeline HITL.
- `useRealtime()`: espone stato canale realtime e sottoscrizioni.

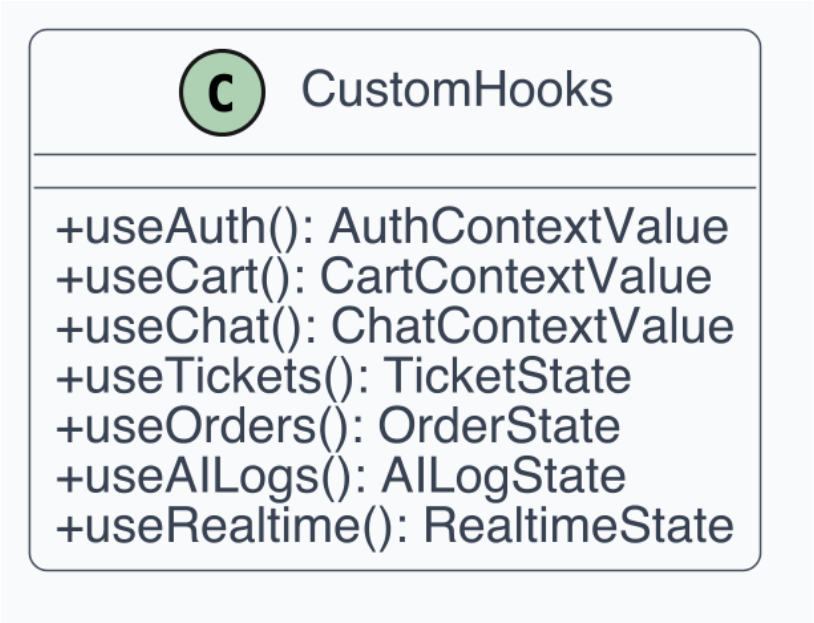

Figura 54: UML del componente `CustomHooks`.

8.2.2.6 RoleBasedRendering **Descrizione del componente:** applica regole di rendering condizionale in base al ruolo, assicurando che le viste operatore non siano esposte ai clienti.
Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma.
Descrizione dei metodi:

- `canRender(role: UserRole, feature: string)`: valuta la visibilita di una feature per un ruolo.
- `renderForRole(role: UserRole)`: compone la porzione UI consentita dal ruolo corrente.

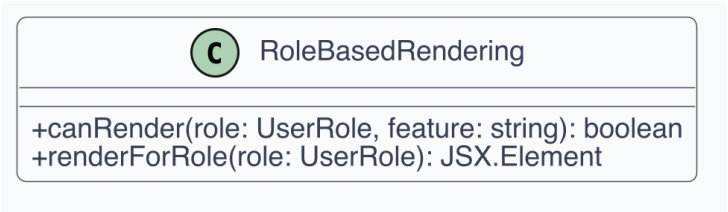

Figura 55: UML del componente `RoleBasedRendering`.

8.2.3 Layer 3 - Service Layer (API Client Modules REST/HTTPS + JWT)

8.2.3.1 authService **Descrizione del componente:** facade di accesso alle API di autenticazione e gestione credenziali.
Descrizione degli attributi: il diagramma non esplicita attributi persistenti.
Descrizione dei metodi:

- `login(credentials: CredentialsDTO)`: esegue autenticazione e recupera sessione/token.

- `logout()`: invalida la sessione lato backend.
- `resetPassword(req: ResetPasswordDTO)`: avvia procedura di reset password.
- `changePassword(req: PasswordChangeDTO)`: aggiorna la password dell'utente autenticato.

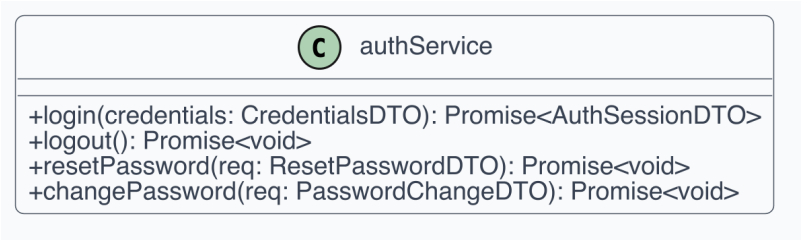

Figura 56: UML del componente `authService`.

8.2.3.2 cartService

Descrizione del componente: modulo API dedicato alle operazioni di carrello e invio ordine, separato dal modulo di sola ricerca catalogo.

Descrizione degli attributi: non sono esplicitati attributi nel diagramma.

Descrizione dei metodi:

- `getCart(sessionId: string)`: recupera lo stato corrente del carrello.
- `upsertCartItem(req: CartItemUpsertDTO)`: inserisce/aggiorna una riga carrello.
- `removeItem(sessionId: string, productId: string)`: rimuove un articolo dal carrello.
- `submitOrder(sessionId: string)`: conferma il checkout e invia l'ordine al backend.

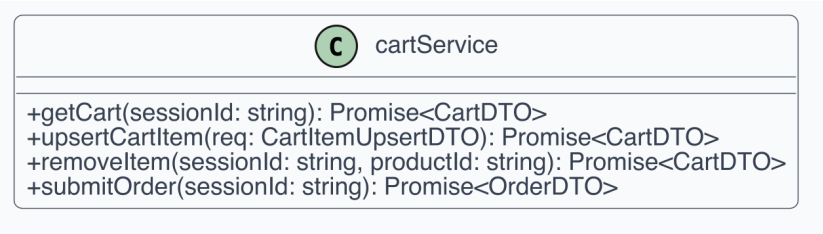

Figura 57: UML del componente `cartService`.

8.2.3.3 productService

Descrizione del componente: modulo read-only per ricerca prodotti a catalogo (live search e filtri assortimento).

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma.

Descrizione dei metodi:

- `searchProducts(query: string, clientId: string)`: interroga catalogo prodotti filtrato sul cliente.

Figura 58: UML del componente `productService`.

8.2.3.4 orderService **Descrizione del componente:** modulo di consultazione ordini gia finalizzati (storico, dettaglio, export CSV).

Descrizione degli attributi: il diagramma non esplicita attributi persistenti.

Descrizione dei metodi:

- `getOrders(filters: OrderFilterDTO)`: recupera lista ordini con filtri e paginazione.
- `getOrderDetail(orderId: string)`: recupera dettaglio completo del singolo ordine.
- `exportCSV(filters: OrderFilterDTO)`: genera export CSV per reporting.

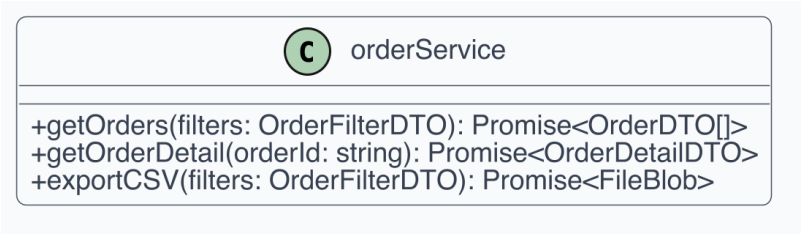

Figura 59: UML del componente `orderService`.

8.2.3.5 ticketService **Descrizione del componente:** modulo API di supporto Human-in-the-Loop per gestione ticket cliente-operatore.

Descrizione degli attributi: non sono rappresentati attributi di stato nel diagramma.

Descrizione dei metodi:

- `getTickets(filters: TicketFilterDTO)`: recupera elenco ticket con filtri operativi.
- `openTicket(req: OpenTicketDTO)`: apre un nuovo ticket di assistenza.
- `closeTicket(ticketId: string)`: chiude un ticket in carico.
- `lockTicket(ticketId: string)`: acquisisce lock operatore sul ticket.

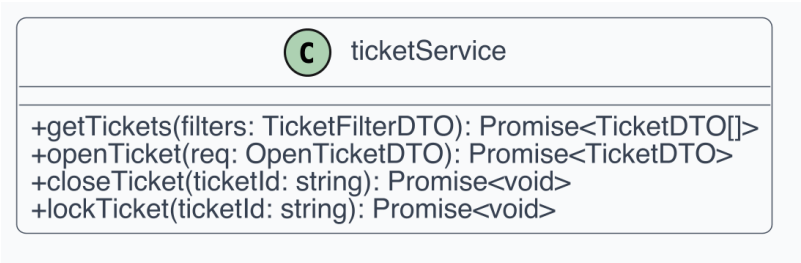

Figura 60: UML del componente `ticketService`.

8.2.3.6 chatService **Descrizione del componente:** facade verso le API conversazionali (messaggi multimodali, storico, feedback AI).

Descrizione degli attributi: il diagramma non esplicita attributi persistenti.

Descrizione dei metodi:

- `sendMessage(req: ChatMessageDTO)`: invia messaggio testo/audio/immagine al backend.
- `getHistory(sessionId: string)`: recupera storico conversazione di una sessione.
- `submitFeedback(req: AIFeedbackDTO)`: invia feedback qualitativo su risposta AI.

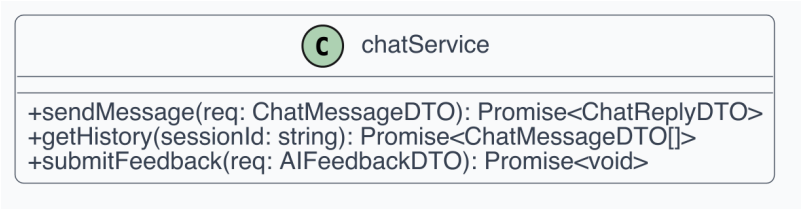

Figura 61: UML del componente `chatService`.

8.2.3.7 feedbackService **Descrizione del componente:** modulo API dedicato al workflow di validazione delle discrepanze AI in dashboard operatore.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma.

Descrizione dei metodi:

- `getLogs(filters: AILogFilterDTO)`: recupera lista log AI filtrata.
- `approveSig(logId: string)`: approva una segnalazione per pipeline HITL.
- `rejectSig(logId: string)`: scarta una segnalazione non valida.
- `revokeSig(logId: string)`: revoca un'approvazione precedente.

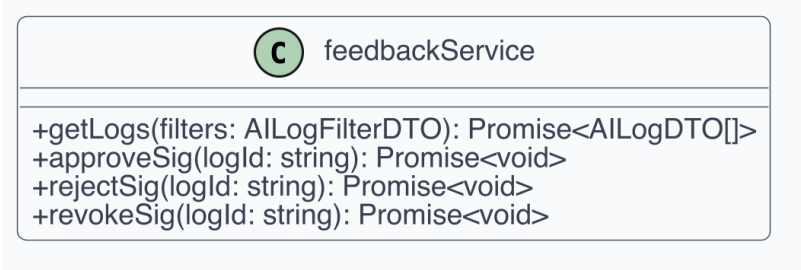

Figura 62: UML del componente `feedbackService`.

8.2.3.8 healthService **Descrizione del componente:** fornisce i moduli per richiamare lo stato di salute del backend, utilissimo per dashboard e logging.

Figura 63: UML del componente `healthService`.

8.2.3.9 sessionService **Descrizione del componente:** separa la verifica di validità della sessione da auth, fornendo hook puliti per il layout client-side.

Figura 64: UML del componente `sessionService`.

8.2.4 Layer 4 - Real-time Layer (WebSocket / SSE Gateway)

8.2.4.1 RTGateway **Descrizione del componente:** gestisce la connessione realtime persistente con backend (WebSocket/SSE), inclusi heartbeat e riconnessione automatica.

Descrizione degli attributi:

- `socketUrl`: endpoint realtime del backend FastAPI.
- `heartbeatMs`: intervallo heartbeat per mantenere viva la connessione.

Descrizione dei metodi:

- `connect(token: string)`: apre la connessione autenticata verso il gateway realtime.
- `reconnect()`: riattiva la connessione in caso di fault di rete.
- `subscribe(topic: string, handler: Function)`: registra un handler per tipologia evento.
- `dispatch(event: RealtimeEvent)`: instrada evento in ingresso all'handler dedicato.

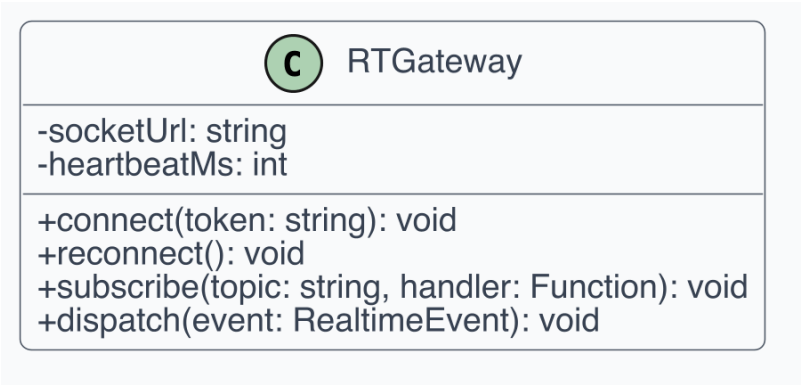

Figura 65: UML del componente `RTGateway`.

8.2.4.2 ChatEventHandler **Descrizione del componente:** elabora eventi chat in tempo reale e aggiorna il contesto conversazionale visualizzato all'utente.

Descrizione degli attributi: il diagramma non esplicita attributi persistenti.

Descrizione dei metodi:

- `handle(event: ChatEvent)`: interpreta l'evento chat ricevuto dal gateway.
- `pushToContext(ctx: ChatContext)`: sincronizza l'evento sullo stato chat condiviso.

Figura 66: UML del componente `ChatEventHandler`.

8.2.4.3 TicketEventHandler

Descrizione del componente: gestisce eventi ticket (apertura, presa in carico, chiusura) e aggiorna dashboard operatore in live sync.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma.

Descrizione dei metodi:

- `handle(event: TicketEvent)`: processa l'evento ticket proveniente dal backend.
- `updateTicketBoard()`: aggiorna la vista tabellare ticket senza reload pagina.
- `notifyOperator(notification: NotificationDTO)`: mostra notifica contestuale all'operatore.

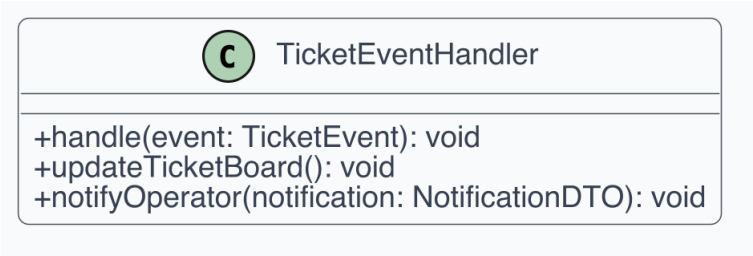

Figura 67: UML del componente `TicketEventHandler`.

8.2.4.4 OrderEventHandler

Descrizione del componente: gestisce eventi di finalizzazione ordine e mantiene allineata la vista ordini globali/storico.

Descrizione degli attributi: non sono rappresentati attributi di stato nel diagramma.

Descrizione dei metodi:

- `handle(event: OrderEvent)`: processa evento ordine in ingresso.
- `refreshOrdersTable()`: aggiorna tabelle ordini in tempo reale.

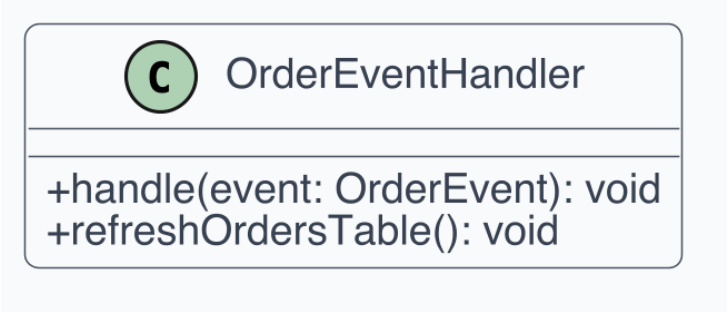

Figura 68: UML del componente `OrderEventHandler`.

8.2.4.5 AILogEventHandler **Descrizione del componente:** presidia la propagazione realtime degli eventi relativi ai log AI e alle azioni di validazione operatore.
Descrizione degli attributi: il diagramma non esplicita attributi persistenti.
Descrizione dei metodi:

- `handle(event: AILogEvent)`: processa evento di segnalazione/cambio stato AI log.
- `refreshAILogsTable()`: aggiorna la vista AI Optimization Dashboard senza reload.

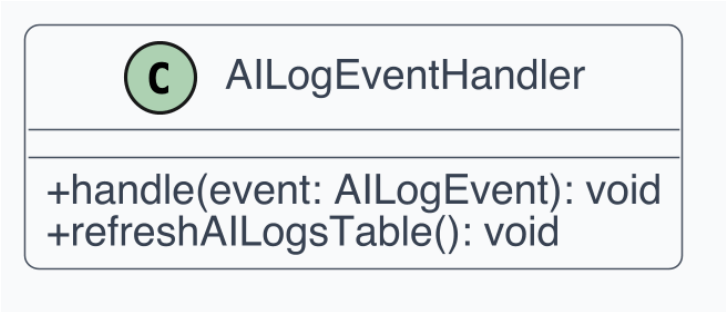

Figura 69: UML del componente `AILogEventHandler`.

8.2.5 Layer 5 - UI Component Library (Atomic Design)

8.2.5.1 Atoms **Descrizione del componente:** contiene componenti visuali minimi e riusabili (es. button, input, badge, spinner) indipendenti dal dominio applicativo.
Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma.
Descrizione dei metodi:

- `buildButton(props: ButtonProps)`: costruisce il componente bottone base.
- `buildInput(props: InputProps)`: costruisce campo input tipizzato.
- `buildBadge(props: BadgeProps)`: costruisce badge informativo/stato.
- `buildSpinner()`: costruisce indicatore visuale di caricamento.

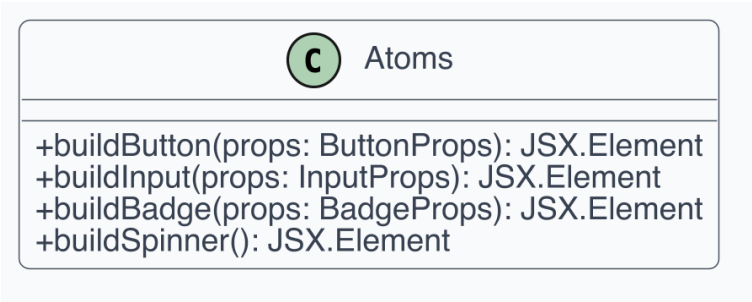

Figura 70: UML del componente `Atoms`.

8.2.5.2 Molecules **Descrizione del componente:** compone piu atoms in widget funzionali di medio livello (es. riga carrello, card prodotto, riga ticket).
Descrizione degli attributi: il diagramma non esplicita attributi persistenti.
Descrizione dei metodi:

- `buildProductCard(data: ProductDTO)`: genera card prodotto per ricerca/catalogo.

- `buildMessageBubble(msg: ChatMessage)`: genera bubble messaggio in chat.
- `buildCartRow(item: CartItem)`: genera riga carrello editabile.
- `buildTicketRow(ticket: TicketDTO)`: genera riga tabellare ticket.

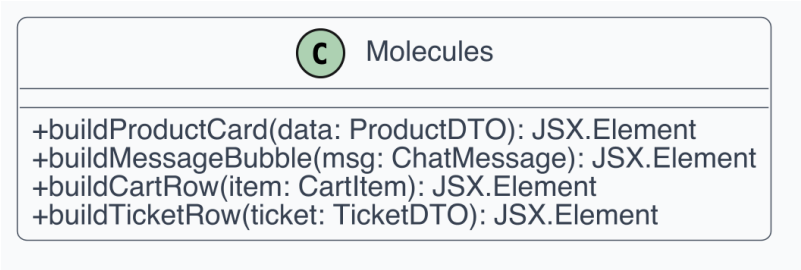

Figura 71: UML del componente `Molecules`.

8.2.5.3 OrganismsCliente **Descrizione del componente:** aggrega molecules/atoms nelle feature di area cliente (chat window, carrello, storico ordini, pannello ricerca).
Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma.
Descrizione dei metodi:

- `renderChatWindow()`: compone area messaggi e input multimodale.
- `renderCartTable()`: compone tabella carrello con azioni sulle righe.
- `renderOrderHistory()`: compone lista storico ordini cliente.
- `renderProductSearchPanel()`: compone ricerca prodotto con risultati live.

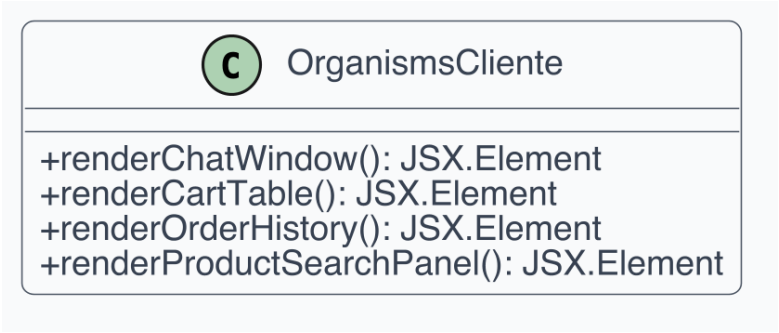

Figura 72: UML del componente `OrganismsCliente`.

8.2.5.4 OrganismsOperatore **Descrizione del componente:** aggrega componenti avanzati desktop-first per operativita backoffice (tab navigator, tabelle, modali di assistenza).
Descrizione degli attributi: non sono rappresentati attributi persistenti nel diagramma.
Descrizione dei metodi:

- `renderTabNavigator()`: gestisce switching tra tab operative.
- `renderTicketTable()`: visualizza ticket con filtri e ordinamenti.
- `renderOrdersTable()`: visualizza ordini globali in tabella.
- `renderAILogTable()`: visualizza segnalazioni AI per validazione.

- `renderTicketModal()`: visualizza modale di assistenza in subentro.
- `renderCartDelegatePanel()`: visualizza il pannello di delega sul carrello cliente durante il subentro operatore.

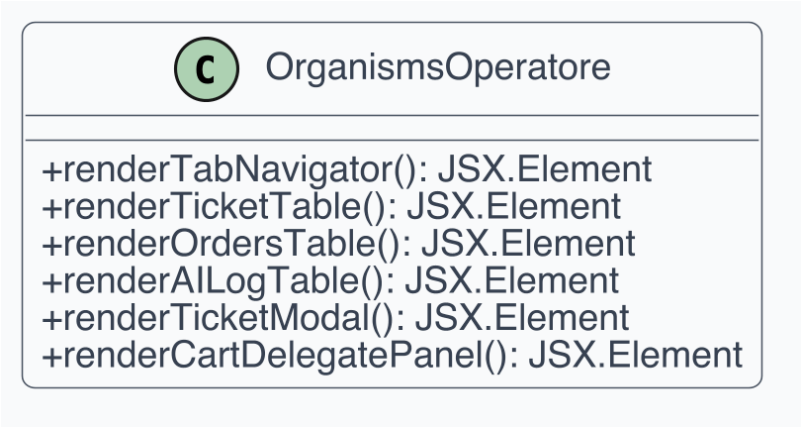

Figura 73: UML del componente `OrganismsOperatore`.

8.2.5.5 SharedUIUtilities **Descrizione del componente:** contiene utility trasversali di presentazione (data table, date picker, dialog e modal) riusate da cliente e operatore.

Descrizione degli attributi: il diagramma non esplicita attributi di stato.

Descrizione dei metodi:

- `renderDataTable(data: any[])`: costruisce tabella con sort, filtri e paginazione.
- `renderDatePicker()`: costruisce selettore intervalli data.
- `openConfirmDialog(req: ConfirmRequest)`: mostra dialog conferma con esito booleano.
- `openModal(component: JSX.Element)`: apre un componente in finestra modale.

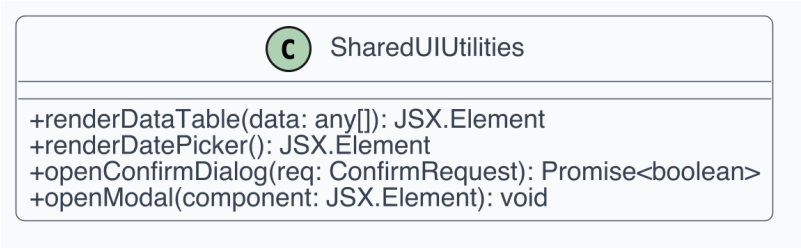

Figura 74: UML del componente `SharedUIUtilities`.

9 Stato dei requisiti funzionali

Questa sezione riporta lo stato di implementazione dei requisiti funzionali derivati dall'ultima versione PB dell'Analisi dei Requisiti (v2.2). Per questa baseline, lo stato di ciascun requisito è indicato nella colonna **Stato**.

ID	Importanza	Descrizione	Stato
`RF-OB_01`	Obbligatorio	Il sistema deve permettere all'Utente di inserire il proprio indirizzo email nel campo dedicato per effettuare il login.	**IMPLEMENTATO**
`RF-OB_02`	Obbligatorio	Il sistema deve permettere all'Utente di inserire la propria password in modo oscurato tramite il campo dedicato.	**IMPLEMENTATO**
`RF-OB_03`	Obbligatorio	Il sistema deve fornire un pulsante o comando per confermare l'accesso e inviare le credenziali per la validazione.	**IMPLEMENTATO**
`RF-OB_04`	Obbligatorio	Il sistema deve mostrare un errore se l'email inserita non ha un formato valido.	**IMPLEMENTATO**
`RF-OB_05`	Obbligatorio	Il sistema deve mostrare un errore se l'email inserita non e presente nel database locale.	**IMPLEMENTATO**
`RF-OB_06`	Obbligatorio	Il sistema deve mostrare un errore se la combinazione di email e password e sbagliata.	**IMPLEMENTATO**
`RF-OB_07`	Obbligatorio	Dopo un login valido, il sistema deve creare la sessione e far accedere l'utente alla propria area (Cliente o Operatore).	**IMPLEMENTATO**
`RF-OB_08`	Obbligatorio	Il sistema deve fornire un pulsante per effettuare il logout, invalidando e chiudendo la sessione corrente lato server.	**IMPLEMENTATO**
`RF-OB_09`	Obbligatorio	Il sistema deve mostrare i dati aziendali del Cliente (ID e Ragione Sociale) in sola lettura.	**IMPLEMENTATO**
`RF-OB_10`	Obbligatorio	Il sistema deve rendere accessibile la pagina Area Personale tramite un elemento di navigazione dedicato, consentendo all'Utente di consultare le informazioni del proprio profilo in base al ruolo assegnato.	**IMPLEMENTATO**
`RF-OB_11`	Obbligatorio	Il sistema deve mostrare all'Utente il proprio indirizzo email in sola lettura.	**IMPLEMENTATO**
`RF-OB_12`	Obbligatorio	Il sistema deve esporre un comando di contatto che, alla sua attivazione, invochi il protocollo URI *mailto:* per aprire il client di posta elettronica nativo dell'Utente con il campo destinatario pre-compilato con l'email di supporto aziendale.	**IMPLEMENTATO**

ID	Importanza	Descrizione	Stato
RF-OB_13	Obbligatorio	Il sistema deve mostrare in tempo reale tutti i messaggi inviati dall'Utente nella chat corrente.	**IMPLEMENTATO**
RF-OB_14	Obbligatorio	Il sistema deve mostrare in tempo reale le risposte testuali scritte dall'AI o dall'Operatore di supporto.	**IMPLEMENTATO**
RF-OB_15	Obbligatorio	Il sistema deve fornire un campo di testo per l'inserimento e l'invio di messaggi testuali nella chat.	**IMPLEMENTATO**
RF-OB_16	Obbligatorio	Il sistema deve bloccare l'invio e mostrare un errore se il messaggio di testo supera il limite di caratteri.	**IMPLEMENTATO**
RF-OB_17	Obbligatorio	Il sistema deve permettere all'Utente la registrazione e l'invio di messaggi vocali tramite il microfono del dispositivo.	**IMPLEMENTATO**
RF-OB_18	Obbligatorio	Il sistema deve convertire l'audio in testo usando un modulo Speech-to-Text.	**IMPLEMENTATO**
RF-OB_19	Obbligatorio	Il sistema deve bloccare l'invio e mostrare un errore se il file audio pesa piu di 10MB.	**IMPLEMENTATO**
RF-OB_20	Obbligatorio	Il sistema deve richiedere esplicitamente i permessi hardware necessari (microfono, fotocamera) al momento del primo utilizzo della relativa funzionalita e, in caso di rifiuto da parte dell'Utente, mostrare un messaggio informativo che spieghi l'impossibilita di procedere senza tali permessi.	**IMPLEMENTATO**
RF-OB_21	Obbligatorio	Il Cliente deve poter resettare la chat con l'AI e svuotare il carrello attuale.	**IMPLEMENTATO**
RF-OB_22	Obbligatorio	Il sistema deve disabilitare il comando di reset della sessione qualora sia presente un ticket di assistenza in stato "Aperto" o "In lavorazione" per la sessione corrente, impedendo all'Utente di azzerare il contesto durante un'assistenza attiva.	**IMPLEMENTATO**
RF-OB_23	Obbligatorio	L'AI (LLM) deve capire quali prodotti vuole l'utente e in che quantita leggendo/ascoltando il messaggio.	**IMPLEMENTATO**
RF-OB_24	Obbligatorio	Il sistema deve cercare i prodotti nel database usando la ricerca vettoriale (Vector Search).	**IMPLEMENTATO**
RF-OB_25	Obbligatorio	Il sistema deve aggiungere in automatico i prodotti trovati al carrello se il Confidence Score calcolato dall'AI supera la soglia definita in RP-OB_03.	**IMPLEMENTATO**

ID	Importanza	Descrizione	Stato
RF-OB_26	Obbligatorio	Vincolo Assortimento: Il sistema deve ignorare i prodotti che non fanno parte del catalogo assegnato a quel Cliente (tabella `asscli`).	**IMPLEMENTATO**
RF-OB_27	Obbligatorio	Vincolo Stato: Il sistema deve impedire l'aggiunta di prodotti segnati come "non disponibili" o "sospesi" nel database (`anaart`).	**IMPLEMENTATO**
RF-OB_28	Obbligatorio	Deduzione Formati: L'AI deve convertire automaticamente le quantita richieste nelle giuste unita di vendita (es. da bottiglie a casse).	**IMPLEMENTATO**
RF-OB_29	Obbligatorio	Il sistema deve fornire una barra di ricerca per cercare manualmente i prodotti nel catalogo tramite input testuale.	**IMPLEMENTATO**
RF-OB_30	Obbligatorio	Il sistema deve mostrare i risultati della ricerca in tempo reale (Live Search).	**IMPLEMENTATO**
RF-OB_31	Obbligatorio	Il sistema deve mostrare per ogni prodotto trovato: Nome, ID, Linea, Famiglia e Modalita di Vendita.	**IMPLEMENTATO**
RF-OB_32	Obbligatorio	L'Utente deve poter cliccare su un risultato per decidere quanti pezzi acquistarne.	**IMPLEMENTATO**
RF-OB_33	Obbligatorio	L'Utente deve poter modificare la quantita usando i tasti +/- o scrivendo direttamente il numero.	**IMPLEMENTATO**
RF-OB_34	Obbligatorio	Il sistema deve correggere in automatico la quantita se l'utente inserisce numeri negativi, zero o troppo grandi.	**IMPLEMENTATO**
RF-OB_35	Obbligatorio	L'Utente deve poter confermare l'aggiunta del prodotto al carrello della sessione corrente.	**IMPLEMENTATO**
RF-OB_36	Obbligatorio	Il sistema deve mostrare un avviso di "Carrello Vuoto" se non ci sono prodotti inseriti.	**IMPLEMENTATO**
RF-OB_37	Obbligatorio	Il sistema deve mostrare i dettagli logistici e le quantita di tutti i prodotti presenti nel carrello.	**IMPLEMENTATO**
RF-OB_38	Obbligatorio	L'Utente deve poter sbloccare la riga di un prodotto nel carrello per poterne modificare la quantita.	**IMPLEMENTATO**
RF-OB_39	Obbligatorio	L'Utente deve poter salvare la nuova quantita modificata per un prodotto nel carrello.	**IMPLEMENTATO**
RF-OB_40	Obbligatorio	L'Utente deve poter rimuovere definitivamente un articolo dal carrello.	**IMPLEMENTATO**

ID	Importanza	Descrizione	Stato
RF-OB_41	Obbligatorio	Il sistema deve disabilitare il pulsante per inviare l'ordine se il carrello e vuoto.	**IMPLEMENTATO**
RF-OB_42	Obbligatorio	L'Utente (Cliente o Operatore in subentro) deve poter chiudere il carrello e inviare l'ordine definitivo.	**IMPLEMENTATO**
RF-OB_43	Obbligatorio	Il sistema deve bloccare l'invio dell'ordine e segnalare gli errori se ci sono problemi con le regole del gestionale (es. prodotto fuori catalogo).	**IMPLEMENTATO**
RF-OB_44	Obbligatorio	Dopo l'invio dell'ordine, il sistema deve svuotare la sessione e reindirizzare l'Utente alla vista appropriata al proprio ruolo: il Cliente allo Storico Ordini, l'Operatore alla chiusura del ticket e dismissione della console di subentro.	**IMPLEMENTATO**
RF-OB_45	Obbligatorio	Il sistema deve creare un file JSON formattato contenente tutti i dati dell'ordine appena chiuso.	**IMPLEMENTATO**
RF-OB_46	Obbligatorio	Il Cliente deve poter visualizzare la lista dei propri ordini passati, ordinati per data.	**IMPLEMENTATO**
RF-OB_47	Obbligatorio	Il sistema deve mostrare per ogni ordine storico: ID, Data, totale degli articoli e una piccola anteprima dei prodotti.	**IMPLEMENTATO**
RF-OB_48	Obbligatorio	Il Cliente deve poter cliccare su un ordine passato per vederne tutti i dettagli logistici.	**IMPLEMENTATO**
RF-OB_49	Obbligatorio	Il sistema deve mostrare in sola lettura la chat originale tra utente e AI che ha generato quell'ordine specifico.	**IMPLEMENTATO**
RF-OB_50	Obbligatorio	Il Cliente deve poter richiedere l'aiuto di un operatore umano, mettendo in pausa l'AI.	**IMPLEMENTATO**
RF-OB_51	Obbligatorio	L'Operatore deve poter accedere alla dashboard con la lista dei Ticket Aperti (richieste d'aiuto).	**IMPLEMENTATO**
RF-OB_52	Obbligatorio	Nella lista dei ticket, il sistema deve mostrare da quanto tempo il Cliente sta aspettando una risposta.	**IMPLEMENTATO**
RF-OB_53	Obbligatorio	L'Operatore deve poter prendere in carico un ticket bloccandolo per gli altri colleghi (Lock) per aprire la console di assistenza.	**IMPLEMENTATO**
RF-OB_54	Obbligatorio	L'Operatore deve poter scrivere e inviare messaggi in chat direttamente al Cliente per fornirgli assistenza.	**IMPLEMENTATO**
RF-OB_55	Obbligatorio	L'Operatore deve poter usare la ricerca per aggiungere forzatamente prodotti al carrello del Cliente che sta aiutando.	**IMPLEMENTATO**

ID	Importanza	Descrizione	Stato
RF-OB_56	Obbligatorio	L'Operatore deve poter modificare le quantita o rimuovere prodotti dal carrello del Cliente.	**IMPLEMENTATO**
RF-OB_57	Obbligatorio	L'Operatore deve poter chiudere il ticket per segnare il problema come risolto e sbloccare la sessione (rimuovere il Lock).	**IMPLEMENTATO**
RF-OB_58	Obbligatorio	L'Utente (Cliente o Operatore) deve poter chiudere il ticket da solo se risolve il problema o non ha piu bisogno di aiuto.	**IMPLEMENTATO**
RF-OB_59	Obbligatorio	L'Operatore deve poter accedere alla dashboard che mostra tutti gli ordini inviati da tutti i Clienti.	**IMPLEMENTATO**
RF-OB_60	Obbligatorio	L'Operatore deve poter cliccare su un ordine di qualsiasi cliente per aprirlo e vederne i dettagli.	**IMPLEMENTATO**
RF-OB_61	Obbligatorio	L'Operatore deve poter cercare del testo solo all'interno di una colonna specifica (es. cercare solo l'ID Ordine).	**IMPLEMENTATO**
RF-OB_62	Obbligatorio	L'Operatore deve poter ordinare le tabelle in modo crescente o decrescente cliccando sui nomi delle colonne.	**IMPLEMENTATO**
RF-OB_63	Obbligatorio	L'Operatore deve poter filtrare i dati delle tabelle scegliendo una data di inizio e di fine.	**IMPLEMENTATO**
RF-OB_64	Obbligatorio	Il sistema deve salvare nel database tutto lo storico delle chat e delle decisioni prese dall'AI.	**IMPLEMENTATO**
RF-OB_65	Obbligatorio	Il sistema deve mostrare la percentuale di sicurezza dell'AI (Confidence Score) solo nelle schermate dedicate all'Operatore.	**IMPLEMENTATO**
RF-OB_66	Obbligatorio	Il sistema deve mostrare schermate di caricamento (spinner o barre di avanzamento) durante le operazioni che richiedono un tempo di elaborazione superiore a 1 secondo.	**IMPLEMENTATO**
RF-OB_67	Obbligatorio	Le API del server (Backend) devono rifiutare qualsiasi richiesta priva di un token di accesso valido per la sessione corrente.	**IMPLEMENTATO**
RF-OB_68	Obbligatorio	Il sistema deve caricare i dati delle tabelle in modo incrementale tramite paginazione con pagine da massimo 50 record, per evitare il blocco dell'interfaccia in presenza di grandi volumi di dati.	**IMPLEMENTATO**

ID	Importanza	Descrizione	Stato
RF-DE_01	Desiderabile	Il sistema deve fornire un link per accedere alla pagina di recupero della password dimenticata.	**NON IMPLEMENTATO**
RF-DE_02	Desiderabile	L'Utente deve poter richiedere il reset della password inserendo la propria email.	**NON IMPLEMENTATO**
RF-DE_03	Desiderabile	Il sistema deve permettere all'Utente loggato di avviare la procedura per cambiare la propria password.	**IMPLEMENTATO**
RF-DE_04	Desiderabile	L'Utente deve poter inviare la vecchia password e la nuova password scelta per salvarla nel sistema.	**IMPLEMENTATO**
RF-DE_05	Desiderabile	Il sistema deve bloccare il cambio password se la vecchia password e errata o se le due nuove password inserite non coincidono.	**IMPLEMENTATO**
RF-DE_06	Desiderabile	Il sistema deve permettere l'invio di file immagine nella chat, mediante scatto fotografico o selezione dalla galleria del dispositivo.	**IMPLEMENTATO**
RF-DE_07	Desiderabile	Il sistema deve bloccare l'invio di immagini e mostrare un errore se il file pesa piu di 5MB.	**IMPLEMENTATO**
RF-DE_08	Desiderabile	Il Cliente deve poter valutare la qualita delle risposte dell'AI lasciando un feedback (es. pollice su / pollice giu).	**IMPLEMENTATO**
RF-DE_09	Desiderabile	Se il Cliente da un voto negativo all'AI, il sistema deve richiedere che indichi il tipo di errore (tramite tag predisposti).	**IMPLEMENTATO**
RF-DE_10	Desiderabile	Il Cliente deve poter aggiungere un testo libero per spiegare meglio il motivo dell'errore segnalato.	**IMPLEMENTATO**
RF-DE_11	Desiderabile	Il Cliente deve poter scaricare l'elenco dei propri ordini storici in un file formato CSV.	**NON IMPLEMENTATO**
RF-DE_12	Desiderabile	L'Operatore deve poter accedere alla dashboard per il controllo degli errori e l'addestramento dell'AI (Ottimizzazione AI).	**IMPLEMENTATO**
RF-DE_13	Desiderabile	Il sistema deve mostrare all'Operatore la lista degli errori segnalati dai clienti e le correzioni manuali fatte ai carrelli.	**NON IMPLEMENTATO**
RF-DE_14	Desiderabile	L'Operatore deve poter aprire un singolo errore AI per rileggere tutta la chat originale tra il Cliente e il bot.	**NON IMPLEMENTATO**
RF-DE_15	Desiderabile	L'Operatore deve poter approvare una segnalazione di errore, confermando che i dati verranno usati per riaddestrare l'AI.	**NON IMPLEMENTATO**

ID	Importanza	Descrizione	Stato
RF-DE_16	Desiderabile	L'Operatore deve poter scartare una segnalazione se non si tratta di un vero errore dell'AI (es. l'utente ha scritto male).	**NON IMPLEMENTATO**
RF-DE_17	Desiderabile	L'Operatore deve poter annullare l'approvazione di una segnalazione se l'aveva confermata per sbaglio (Rollback).	**NON IMPLEMENTATO**
RF-OP_01	Facoltativo	Il sistema deve permettere l'invio di messaggi d'ordine e la loro elaborazione tramite l'app di WhatsApp.	**NON IMPLEMENTATO**
RF-OP_02	Facoltativo	Il sistema deve permettere l'invio di messaggi d'ordine e la loro elaborazione tramite l'app di Telegram.	**NON IMPLEMENTATO**
RF-OP_03	Facoltativo	Il sistema deve generare grafici e statistiche generali sull'uso della piattaforma, mostrandoli all'Operatore.	**IMPLEMENTATO**

Tabella 8: Stato dei requisiti funzionali (fonte: Analisi dei Requisiti PB v2.2)

9.1 Riepilogo stato requisiti prestazionali e di vincolo

Categoria	Totale	Implementati	Non implementati
RP-OB	5	5	0
RP-DE	1	0	1
RV-OB	10	10	0

Tabella 9: Riepilogo implementazione requisiti prestazionali e di vincolo